



武汉大学

WUHAN UNIVERSITY

全国大学生计算机系统能力大赛操作系统
OS 功能挑战赛

Aegis-NX——NuttX OS Heap 异常检测工具 设计方案文档

参赛学校： 武汉大学

项目成员： 刘子俊 袁孜琪 周晓璐

指导教师： 龚奕利

二〇二五年六月

摘要

本项目为 2025 年全国大学生计算机系统能力赛-操作系统设计赛-OS 功能挑战赛道的 248 号赛题：Heap 异常检测工具。针对 NuttX 实时操作系统在工业控制、无人机等高可靠性场景中面临的严峻内存安全挑战，本项目设计并实现了一套轻量级、集成化的堆异常检测方案：Aegis-NX。方案旨在有效侦测并辅助定位两类核心内存问题：即时性的内存安全违例与累积性的内存泄漏，以提升 NuttX 系统的健壮性与安全性。

项目主要工作分为以下两个部分：

1.即时性内存错误检测 (KFENCE)：为高效捕获堆缓冲区溢出 (Heap Buffer Overflow)、释放后使用 (Use-After-Free) 和重复释放 (Double-Free) 等严重错误，我们实现了一个受 Linux KFENCE 启发的功能模块。该工具基于采样和硬件防护页 (Guard Page) 机制，以极低的性能开销运行，使其能够在开发乃至发布版本中持续使用。一旦捕获到异常，系统能提供包含调用栈在内的精确诊断信息，帮助开发者快速定位问题根源。

2.累积性内存泄漏检测 (HMMLEAK)：针对内存泄漏的特征，我们设计并实现了一套创新的、基于概率性风险评估的检测工具 HMMLEAK。该工具的核心是一种多特征加权评分模型，它将内存泄漏检测从一个确定性问题转化为风险评估问题。HMMLEAK 通过 Hook 内存管理函数，在 NuttX 中首次实现了对各进程内存分配行为的独立追踪与分析。

本项目的主要特色与创新点如下：

1.智能采样策略：为提升 KFENCE 的检测效率，我们引入了动态可调的采样间隔与 Counting Bloom Filter 过滤器。该策略能智能地避免对同一代码路径的重复低效采样，从而将检测资源更有效地分配到系统的各个角落。

2.混合式警戒区防护：在内存保护方面，我们采用了粗粒度与细粒度相结合的警戒区 (Red Zone) 技术。这种混合方法能够在保证高精度检测（如捕获单字节溢出）的同时，最小化性能开销，实现了准确性与运行效率的平衡。

3.硬件平台双模支持：针对 NuttX 生态中硬件平台多样化的特点，KFENCE 在设计上实现了对有/无内存管理单元 (MMU) 硬件的双重支持。对于无 MMU 的主流微控制器，它采用高效的软件“警戒区” (Red Zone) 机制；而对于具备

MMU 的平台，则能够利用硬件“防护页”(Guard Page) 实现更高性能的隔离与保护，展现了方案的广泛适应性。

4.创新的 HMMLEAK 风险评估模型：针对内存泄漏的特征，我们首创了一套基于概率的风险评估模型。该模型通过内存泄漏的多个特征采用多维度分析器为进程内存行为画像，并根据可配置的权重计算出最终内存泄漏风险分值。

5.高效的 Per-Process 追踪架构：为弥补 NuttX 当前欠缺按进程分析内存的能力，我们设计了由三级哈希表构成的追踪架构。该架构通过高效的地址索引，将被动触发的内存块查找操作的平均时间复杂度控制在 $O(1)$ 级别，确保了低开销与数据一致性。

6.预定义多场景权重模型 (HMMLEAK)：为提升 HMMLEAK 的易用性，我们设计并实现了一套预定义的特征权重模型，包含均衡、高吞吐、长时任务等多种模式。用户可根据应用特点一键切换配置，而无需手动调整复杂的权重参数，显著降低了使用门槛。

7.引入工业级安全测试标准：项目在保持 NuttX 测试生态一致性的前提下，首次将工业级安全测试标准 Juliet 引入 RTOS 领域，为 KFENCE 在资源受限设备中的可靠性提供了学术与实践的双重验证，显著提升 NuttX 在安全敏感场景（如 IoT/工控）的部署置信度。

8.硬件验证部署：尤其关键的是，本方案已成功移植并在 BeagleBone Black 真实硬件平台上完成部署与验证，结果表明 Aegis-NX 能够在典型的工业级嵌入式环境中稳定运行，并以可控的性能开销实现有效的堆异常检测，充分证明了其在实际应用中的可行性与价值。

目录

1. 绪论	5
1.1 项目背景与意义	5
1.2 问题定义与挑战	6
1.3 设计目标	7
1.4 组织结构	7
2. 总体设计方案	9
2.1 开发环境	9
2.2 总体架构	9
2.3 设计原则	11
3. KFENCE-及时性内存异常检测	12
3.1 设计理念	12
3.2 功能实现	12
3.3 异常检测	17
3.4 配置选项	18
4. HMMLEAK-内存泄漏检测	19
4.1 设计理念与架构	19
4.2 核心数据结构	20
4.3 数据追踪层	23
4.4 特征分析层	25
4.5 决策评分引擎	29
4.6 报告与控制层	32
5. 硬件平台移植与实现	35
5.1 硬件平台选型与介绍	35
5.2 系统移植与适配过程	36
5.3 部署环境展示	38
6. 测试与评估	39
6.1 概述	39
6.2 测试环境	39
6.3 KFENCE 功能测试	40
6.4 HMMLEAK 功能测试	50
6.5 性能测试	53
7. 开发中遇到的问题与解决方案	58
7.1 KFENCE 开发	58
7.2 HMMLEAK 开发	58
7.3 测试	59
7.4 硬件平台移植	61
8. 总结与展望	62
8.1 项目总结	62
8.2 项目展望	63
参考文献	64
附录 A: 项目与外部资源关系说明	65

1. 绪论

1.1 项目背景与意义

内存安全是构建可靠与安全软件系统的基石。然而，在以 C/C++ 等无内存安全保障的语言构建的系统中，内存错误始终是威胁系统稳定性的一个核心隐患。根据微软和谷歌等业界巨头的公开报告，其产品中高达 70% 的安全漏洞都与内存安全问题直接相关。这些漏洞一旦被利用，攻击者便可轻易实现数据泄露、篡改关键数据，乃至获取远程代码执行权限，并发动如 ROP、JOP 等精密复杂的攻击。这类问题通常可分为两大类：一是空间安全违例，如数组的越界访问；二是时间安全违例，如臭名昭著 Use-After-Free 或 Double-Free。

与此同时，对于未使用自动垃圾回收机制的语言，内存泄漏 (Memory Leaks) 也是一个长期存在的严重问题。它源于程序对系统内存资源非必要的长期占用，虽然不会像内存破坏那样立即导致崩溃，但其累积效应会持续增加系统负载、降低响应速度，最终导致性能的严重恶化甚至服务中断。这种“温水煮青蛙”式的资源耗尽，因其难以在有限的开发测试周期和多变的工作环境中被稳定复现，往往成为最棘手的软件缺陷之一。

上述内存安全问题在通用计算领域已是沉痾宿疾，而在嵌入式与实时操作系统 (RTOS) 的世界里，其影响则更为深远和严峻。NuttX，作为一款高度可定制、兼容 POSIX 标准的开源 RTOS，正凭借其灵活性与高性能，广泛应用于工业控制、无人机飞控、高端物联网网关等对长期稳定性和高可靠性有极致要求的场景。在这些场景中，系统运行的环境与通用服务器或 PC 截然不同，其固有的特性使得内存问题的影响被急剧放大。在这些严苛的应用场景下，系统的可靠性直接关系到物理世界的安全，一次内存越界或释放后使用，其后果便不再局限于程序崩溃，而可能直接导致物理设备的失控或关键数据的永久性损坏。更为严峻的是，NuttX 所运行的微控制器 (MCU) 环境，其 RAM 和闪存空间又极其宝贵，这使得内存泄漏的累积效应变得尤为致命，它会迅速耗尽系统的生命线，同时也让重量级的传统检测工具因其自身巨大的资源开销而毫无用武之地。不仅如此，系统对实时性的根本要求，决定了任何解决方案都必须拥有极低且可预测的性能开销，否则便可能干扰关键任务的调度，错过响应时限 (deadline)，而这恰恰是嵌入式系统最需要避免的。

因此，在 NuttX 生态中，一个能够在不牺牲实时性、不引入显著性能开销的前提下，有效侦测并辅助定位上述两类内存问题的工具，已不再是“锦上添花”，而是保障系统可靠性的“雪中送炭”。本项目旨在面向 NuttX 操作系统的独特挑战，设计并实现一套轻量级的堆异常检测方案——Aegis-NX。这不仅能帮助开发者在早期阶段以更低成本修复常见的编程错误，更能作为一道坚固的防线，提升最终部署产品的健壮性与安全性。

1.2 问题定义与挑战

本项目聚焦于解决 NuttX 环境下两类核心的堆内存异常：一类是能被即时捕获的、破坏性的内存安全违例；另一类是效应随时间累积的内存资源泄漏。我们的目标是为这些定义清晰的异常模式提供精确、高效的检测方案。

1.2.1 内存安全违例 (Memory Safety Violations)

这类错误通常源于对内存的非法访问或对管理函数的误用，会立即或在短时间内破坏程序状态，是导致系统崩溃和安全漏洞的直接原因，我们将此类问题进一步细化为：

(1) 堆缓冲区溢出 (Heap Buffer Overflow): 这是最常见的内存错误。它既可能表现为大幅度的越界，直接访问到被硬件保护的防护页 (Guard Page)；也可能是小范围的、潜伏的溢出，仅污染到对象旁我们预设的警戒区 (Red Zone)。两者都必须被有效检出。

(2) 释放后使用 (Use-After-Free): 在 free 调用之后，程序仍通过悬空指针读写该内存区域。

(3) 重复释放 (Double-Free): 程序对同一内存对象执行了两次或以上的 free 操作。

(4) 无效的释放/重分配目标: 尝试 free 或 realloc 一个并非由 malloc 家族函数分配的指针（如栈地址、全局地址或随机值）。

(5) 错误的释放/重分配地址: 传入的指针地址虽然有效，但并非内存对象的起始地址（例如，对 `addr+1` 进行释放）。

1.2.2 内存泄漏 (Memory Leaks)

与上述能被即时捕获的错误不同，内存泄漏是程序在动态申请内存后，因逻辑疏忽而未能释放，导致内存资源被无限期占用。在 NuttX 这类需要长期稳定运行的嵌入式系统中，内存泄漏是“慢性毒药”，它会逐渐侵蚀系统有限的内存资源，

最终导致性能下降、功能异常乃至系统完全崩溃。为了有效检测上述问题，我们必须直面以下核心挑战：

（1）性能与精度的权衡：这是一个长期存在的核心挑战。最精确的检测（例如对每一次内存访问都进行插桩检查）带来的性能损耗是嵌入式系统无法承受的。如何在可接受的性能开销（例如低于 5%）下，实现尽可能高的异常捕获率，是设计的关键。

（2）问题的定位与归因：仅报告“发生了内存泄漏”是远远不够的。如何从海量的内存操作中，准确识别出那些“只进不出”的可疑分配源头？这要求工具能有效地追踪每个分配点的行为特征，清晰地报告出内存泄漏可能性较大的进程，并提供其累计分配次数、总大小等量化证据以及其中泄漏可能性较大的内存块，以便快速锁定“头号嫌疑人”。

1.3 设计目标

为应对上一节所定义的挑战，并有效解决特定的内存异常问题，本项目旨在 NuttX 操作系统中设计并实现一个集成的、轻量级的堆异常检测系统——Aegis-NX。具体设计目标如下：

（1）实现一个用于即时性内存错误检测的 KFENCE 工具。基于采样和硬件防护页机制，能够精确、高效地捕获包括 Buffer Overflow、UAF、Double-Free 在内的关键错误。其设计必须追求极低的性能开销，以保证在开发乃至发布版本中持续使用的可行性，并在捕获异常时提供包含调用栈的精确诊断信息。

（2）设计并实现一个用于内存泄漏检测的 HMMLEAK 工具。该工具的核心是一种创新的、基于多特征的加权评分模型。它能够持续追踪各分配点的行为，并根据风险评分进行预警。为适应多样的应用场景，该模型的特征与权重应是易于配置的。同时，检测应支持按进程进行，以弥补当前 NuttX 在此方面的不足。

（3）提供一套完整的、与 NuttX 内核无缝集成并能在硬件上充分验证的解决方案。这要求两个工具不仅功能独立，还要能平滑地集成到 NuttX 的内存管理子系统中，并为用户提供清晰的配置与交互接口。同时，我们将通过专门设计的 benchmark 对工具的有效性和性能开销进行充分的验证与评估。

1.4 组织结构

本文的组织结构安排如下，旨在系统性地阐述我们的研究工作：

第一章：绪论。介绍项目的研究背景与意义，明确定义了需要解决的内存异常问题与核心挑战，并提出了本项目的设计目标。

第二章：总体设计方案。阐述项目的顶层架构，说明 KFENCE 和 HMMLEAK 两个子系统如何协同工作，并与 NuttX 内核进行交互。

第三章：KFENCE 的设计与实现。详细介绍用于即时性错误检测的 KFENCE 工具，深入其采样机制、防护页原理、元数据设计以及各类异常的捕获逻辑。

第四章：HMMLEAK 的设计与实现。详细介绍用于内存泄漏检测的 HMMLEAK 工具，重点阐述其创新的多特征加权评分模型、及可配置的预警机制。

第五章：硬件平台移植与实现。阐述将 Aegis-NX 方案从仿真环境移植到 BeagleBone Black 物理硬件平台的完整过程，包括平台选型、系统适配、环境部署与验证效果，突出项目在真实物理环境中的实践成果。

第六章：测试与评估。展示在物理硬件平台上的测试环境、功能验证方法以及详尽的性能评估结果，通过数据证明我们实现的工具兼具有效性与低开销特性。

第七章：开发中遇到的问题与解决方案。记录和总结在项目研发全周期中遇到的关键技术难题以及我们团队所采取的解决方案。

第八章：总结与展望。对本项目当前完成的工作进行总结，并对未来可能的改进方向和扩展功能进行展望。

2. 总体设计方案

2.1 开发环境

本项目的开发与验证采用了仿真与物理硬件相结合的双平台策略。初期功能开发与无 MMU 场景的验证在 QEMU 仿真环境中进行，而最终的功能、性能与稳定性验证则在 BeagleBone Black 物理硬件平台上完成。具体环境配置如下：

- 主机操作系统:Ubuntu 20.04 LTS (64-bit)
- 编译工具链 :GNU Arm Embedded Toolchain
- 目标操作系统:NuttX RTOS

平台一：QEMU 仿真环境 (用于初期开发与无 MMU 场景验证)

- 模拟器: QEMU 模拟 stm32-p103 开发板
- 目标处理器: ARM Cortex-M3 (基于 ARMv7-M 架构, 无 MMU)
- 模拟资源: 20KB SRAM, 64KB Flash
- 主要用途:
 - 快速进行 Aegis-NX 核心算法的原型开发与迭代。
 - 验证 KFENCE 在无 MMU 硬件环境下的软件“警戒区”(Red Zone)检测机制的有效性。

平台二：BeagleBone Black 物理验证平台 (用于最终验证与 MMU 场景测试)

- 开发板型号: BeagleBone Black Rev C
- 目标处理器: TI Sitara AM3358 (1GHz ARM Cortex-A8, 带 MMU)
- 硬件资源: 512MB DDR3 RAM, 4GB eMMC Flash
- 主要用途:
 - 验证 Aegis-NX 方案在真实物理硬件上的可移植性、稳定性和可靠性。
 - 测试并验证 KFENCE 利用 MMU 实现的硬件“防护页”(Guard Page)机制。
 - 进行精确的性能基准测试 (CoreMark, Dhrystone), 评估方案在实际应用中的性能开销。

2.2 总体架构

Aegis-NX 的堆异常检测系统作为 NuttX 内核的一个集成增强模块,其整体架构设计旨在实现高效、全面且对系统侵入性最小的内存监控。该系统与 NuttX 的核心内存管理器(mm_heap)紧密协作,能够在不改变上层应用代码的前提下,透

明地感知每一次堆内存操作。

该架构由两个功能互补的子系统 KFENCE 和 HMMLEAK 组成，整体架构如下图所示：

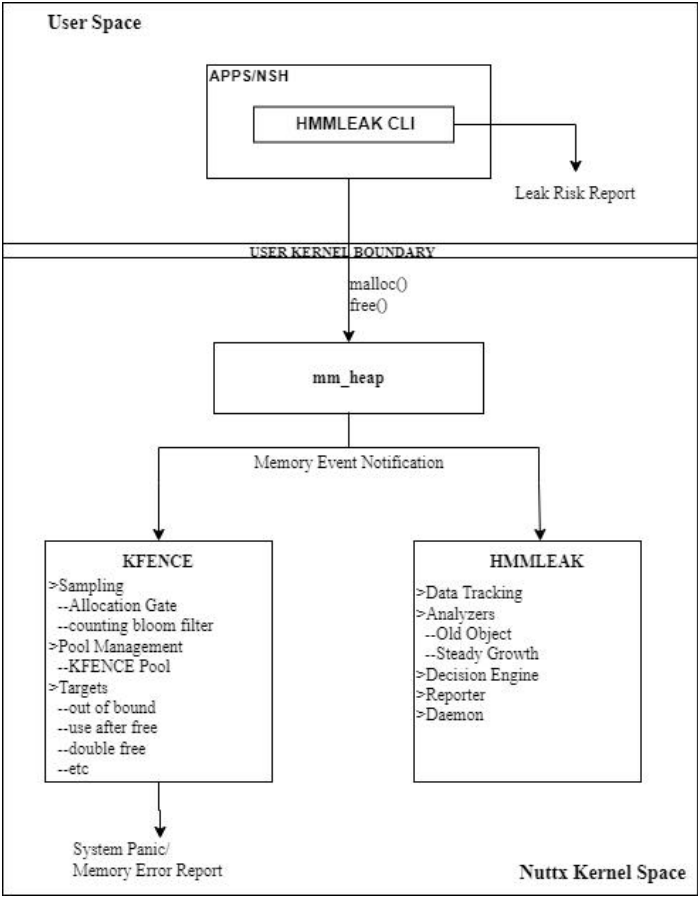


图 2.1 系统架构图

其关键组成部分与工作流程如下：

(1) 交互机制：当系统中有任何代码请求分配或释放堆内存时，这些操作都会被内存管理器捕获。通过在内存管理流程中建立的通知机制，相关的事件信息（如指针、大小、进程 ID 等）会被分发至我们的检测系统，从而启动相应的分析与处理流程。

(2) 即时性内存错误检测器：KFENCE 子系统。KFENCE 专注于以极低的性能开销实时捕获破坏性的内存安全违例，如堆缓冲区溢出、释放后使用和重复释放等。对于每个内存分配请求，KFENCE 的采样逻辑会首先进行评估。若该请求被选中，它将从一个两侧设有“警戒区”(Red Zone)的特殊内存池中分配，以实现严密监控。若未被选中，则按常规流程分配，性能开销极低。

(3) 累积性泄漏分析器：HMMLEAK 子系统。HMMLEAK 专注于识别模式更

隐蔽、效应随时间累积的内存泄漏问题。HMMLEAK 子系统会记录每一次成功的堆内存分配与释放事件，并在其内部高效的、按进程(PID)组织的追踪数据结构中更新对应信息。其核心分析引擎在后台周期性或按需运行，采用一套创新的多特征加权评分模型，从对象老化、内存占用稳定增长等多个维度为每个进程的内存行为进行风险评估。

2.3 设计原则

Aegis-NX 的总体设计旨在为 NuttX 操作系统提供一个强大的堆内存异常检测能力，其实现遵循以下核心原则，以确保方案的有效性、高效性和实用性，满足实时操作系统在安全关键领域中的严苛标准。

(1) 低开销与高性能原则 (Low Overhead & High Performance)

任何监控和检测机制的引入都必须将对系统性能的影响降至最低。该工具的设计追求极致的轻量化，确保其在运行时不会显著增加 CPU 负载或内存占用，更不能影响系统的实时响应能力，使其能够在开发乃至发布版本中持续使用。

(2) 健壮性与安全原则 (Robustness & Security)

设计的首要目标是提升 NuttX 系统的整体健壮性与安全性。通过主动、有效地侦测并报告各类堆内存异常，旨在构筑一道坚固的防线，防止因内存错误导致的程序崩溃、性能下降以及潜在的安全漏洞被利用。

(3) 精准诊断与易用性原则 (Precise Diagnostics & Usability)

发现问题只是第一步，帮助开发者快速定位和解决问题同样至关重要。当检测到异常时，系统必须提供清晰、准确且信息丰富的诊断报告，如精确的错误类型和关键的上下文信息（如调用栈），以最大程度地降低调试成本和复杂度。

(4) 高可配置与灵活性原则 (High Configurability & Flexibility)

为了适应多样的应用场景和不同的开发阶段，本方案必须具备高度的灵活性。检测工具的核心参数，如检测的敏感度、分析策略的权重等，应设计为易于配置的选项，允许开发者根据实际需求进行调整和优化。

(5) 无缝集成原则 (Seamless Integration)

Aegis-NX 应作为 NuttX 操作系统的有机组成部分，而非一个孤立的外部工具。其设计应能平滑地集成到 NuttX 的内核子系统中，与现有架构无缝协作，保证系统的整体性和稳定性。

3.KFENCE-及时性内存异常检测

3.1 设计理念

KFENCE（Kernel Electric-Fence）是一种低开销的内存错误检测机制，其核心理念是通过抽样和警戒区保护实现对内存错误的实时捕捉。

与 KASAN 等传统的重量级内存检测工具不同，KFENCE 基于采样检测，通过周期性开启分配门（`g_allocation_gate`）的方式，仅对部分内存分配请求进行检测，将这部分内存分配到 KFENCE 专属的内存池中，大幅降低运行时开销，实现了性能与检测能力的平衡。同时，通过分配栈哈希和 Counting Bloom Filter 等技术，提高采样的多样性和覆盖率。

为了适应不同的硬件需求，对于有 MMU 和无 MMU 的两种情况，我们采用了不同的方法实现 KFENCE。对于有 MMU 的系统，利用硬件页保护机制，通过警戒页（Guard Page）实现内存隔离；对于无 MMU 的系统，则采用软件的方式检测内存访问。

3.2 功能实现

3.2.1 内存池管理

KFENCE 的内存池管理主要由三个部分组成：预分配的固定大小内存池、元数据数组、空闲链表。

（1）内存池

KFENCE 的内存池由若干个大小固定的内存块组成，每个内存块两侧都设有警戒区，用于检测越界访问。

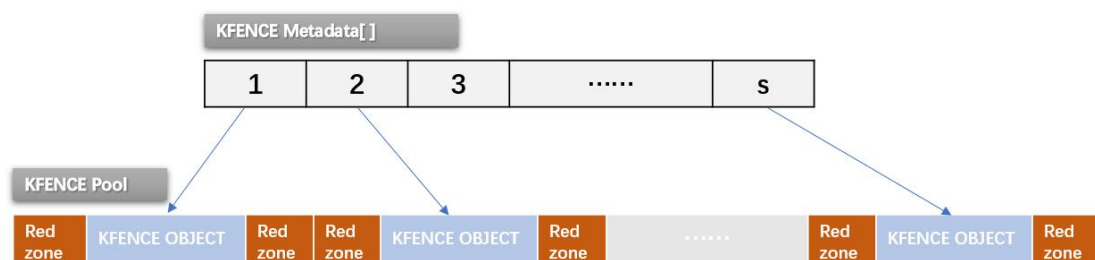


图 3.1 KFENCE 内存池示意图（无 MMU）

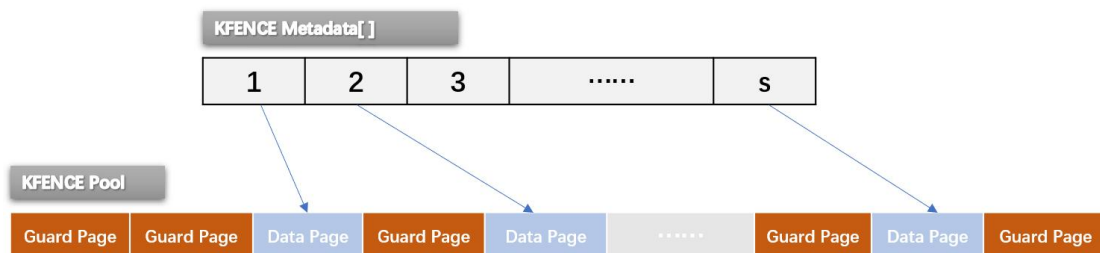


图 3.2KFENCE 内存示意图（有 MMU）

（2）元数据数组

元数据（`kfence_metadata_s`）是 KFENCE 的核心数据结构，记录每个采样对象的完整信息：

```
struct kfence_metadata_s
{
    FAR struct kfence_metadata_s* next; // 空闲链表指针
    spinlock_t lock; // 保护元数据的自旋锁
    enum kfence_object_state_e state; // 对象状态
    uintptr_t addr; // 分配的内存地址
    size_t size; // 分配的大小
    FAR struct mm_heap_s* heap; // 所属堆
    struct kfence_track_s alloc_track; // 分配追踪信息
    struct kfence_track_s free_track; // 释放追踪信息
    uint32_t alloc_stack_hash; // 分配栈哈希值
#ifdef CONFIG_ARCH_USE_MMU && defined(CONFIG_MM_PGALLOC)
    void* vaddr; // 对象虚拟地址
    uintptr_t paddr; // 对象物理页
    void* guard_vaddr; // 警戒页虚拟地址
    uintptr_t guard_paddr; // 警戒页物理页
#endif
};

struct kfence_track_s
{
    pid_t pid; // 执行操作的进程 ID
    int cpu; // 执行操作的 CPU
    uint64_t ts_nsec; // 时间戳（纳秒）
    size_t num_stack_entries; // 有效的堆栈条目数量
    void *stack_entries[KFENCE_STACK_DEPTH]; // 堆栈跟踪地址
};
```

（3）空闲对象链表：用于连接每一个空闲 KFENCE 内存对应的元数据对象。

3.2.2 采样机制

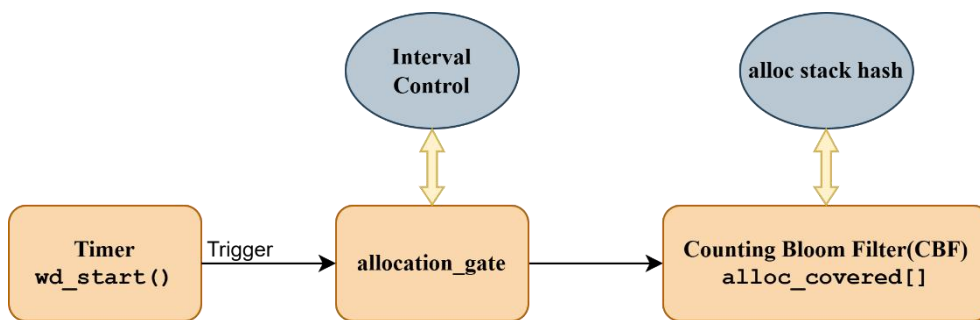


图 3.3 KFENCE 采样机制示意图

整个采样控制系统采用三级架构设计。最外层是分配门控机制，通过分配门（`g_allocation_gate`）实现令牌桶算法控制采样频率。系统初始化时设置初始值为 1 的分配门，定时器触发时将其设为负值，允许接下来几次分配被采样。每次内存分配时计数器递增，正值则跳过采样，负值或零进入采样流程。这种设计确保采样间隔均匀分布，避免性能波动。中间层是定时调节系统，动态调整采样参数，包括基础采样间隔（默认 100ms 可调）、负载自适应机制（高负载时延长间隔）和错误反馈调节（检测到错误时提高相关路径采样率）。最内层是智能过滤系统，根据分配调用栈（`alloc_track->stack_entries`）计算该分配点哈希值，并通过计数式布隆过滤器（Counting Bloom Filter）过滤，避免重复采样相同代码路径，确保采样多样性。

当应用程序发起内存分配时，KFENCE 首先进行快速检查，过滤掉过大尺寸的请求。然后通过分配门判断是否进入采样路径，接着分析调用栈计算特征哈希，最后通过智能过滤系统综合决策。

函数名	输入	说明
<code>uint32_t stack_trace_hash</code>	<code>stack_entries, num_entries</code>	计算分配栈哈希值
<code>bool should_skip_covered</code>	None	判断是否启用 CBF 过滤采样
<code>void alloc_covered_add</code>	<code>alloc_stack_hash, val</code>	在 CBF 中为对应位置增加计数（1 或-1）
<code>bool alloc_covered_contain</code>	<code>alloc_stack_hash</code>	查找 CBF 中哈希值对应位置是否被占用
<code>void toggle_allocation_gate</code>	None	实现周期性采样

表 3.1 采样机制相关函数说明

3.2.3 警戒区保护

KFENCE 的警戒区保护机制是其检测内存错误的核心技术之一，通过在分配内存周围设置特殊标记区域来捕捉越界访问。KFENCE 对象在分配时，通过 `set_redzone(struct kfence_metadata_s *metadata)` 函数在实际分配的内存块前后预留了警戒区（Red Zone），这些区域填充了特定模式的 canary 值。当内存被释放或进行边界检查时，系统会验证对应 canary 值是否被修改，以此判断是否发生了越界访问。

警戒区的 canary 值设计是保护机制的关键所在，KFENCE 设计了两种模式。通过与地址低 3 位异或，使得每个 8 字节对齐区间内的 canary 值都不同，这增加了攻击者预测或绕过保护的难度；在保持每个 8 字节块内 canary 值变化的同时，允许使用 64 位宽内存操作一次性设置或检查 8 字节数据，大幅提高了处理效率。

CANARY PATTERN	VALUE
CANARY_PATTERN_U8	0xaa ^ (addr & 0x7)
CANARY_PATTERN_U64	0xaaaaaaaaaaaaaaaa ^ 0x0706050403020100

表 3.2 警戒区 CANARY 模式说明

在设置警戒区时，系统首先计算内存块边界，然后使用模式填充。对于对齐的地址区域，采用 64 位写操作批量填充 CANARY_PATTERN_U64；对于非对齐部分则使用逐字节填充 CANARY_PATTERN_U8。这种混合策略既保证了处理速度又确保了全面覆盖。检查警戒区时同样采用类似策略，先快速扫描 64 位块（ `check_redzone(struct kfence_metadata_s *metadata)` ），发现异常后再调用 `set_redzone_byte(uint8_t *addr)` 细粒度检查具体字节。这种分层检查方法显著减少了正常情况下的性能开销。

函数名	输入	说明
void set_redzone	metadata	在分配内存块前后设置警戒区
void check_redzone	metadata	检查警戒区是否被破坏
bool check_redzone_byte	(uint8_t *)addr	逐字节查找被污染的内存

表 3.3 警戒区相关函数说明

3.2.4 内存分配

在 KFENCE 分配路径中,系统首先从全局空闲链表 `g_kfence_free_list` 获取一个元数据块,这个链表管理着所有可用的 KFENCE 内存对象。获取元数据后,系统会调用 `kfence_update_metadata` 初始化元数据信息,并记录分配时的调用栈,用于后续错误诊断。实际的内存地址来自预分配的 `g_kfence_pool` 内存池,这个池子被划分为固定大小的槽位,每个槽位包含实际内存和两侧的警戒区。

在内存成功分配后,会调用 `alloc_covered_add` 使 CBF 中对应位置计数自增,并且更新全局统计信息,包括增加已分配对象计数等。整个过程通过细粒度锁(如 `metadata->lock`) 保护关键数据结构,确保多线程环境下的正确性。最终返回给用户被保护的内存区域,任何越界访问或非法使用都会被立即检测到。

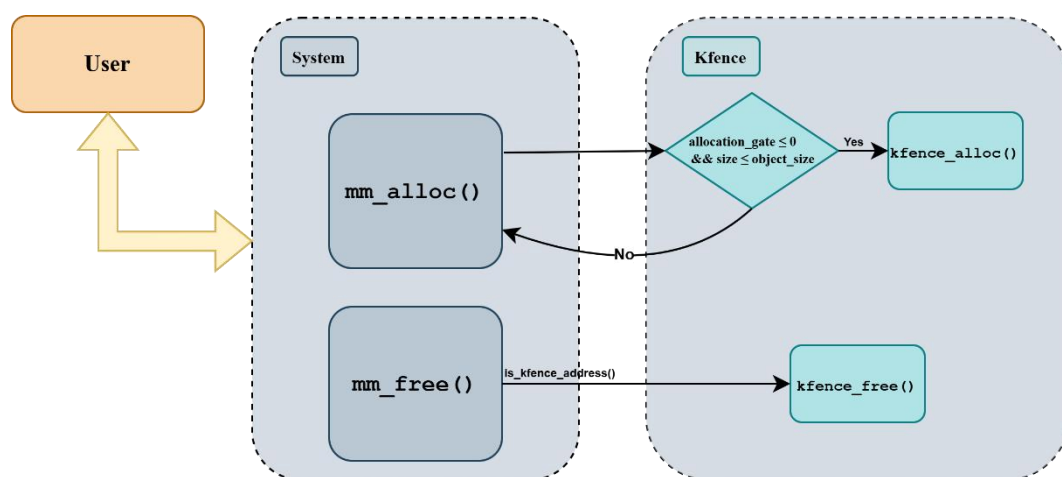


图 3.4 KFENCE 内存分配/释放流程图

3.2.5 内存释放

KFENCE 的内存释放机制实现了一套严格的状态验证和错误检测流程,与内存分配过程紧密配合形成完整的安全防护体系。当应用程序调用 `free` 释放内存时,如果检测到请求释放的内存地址属于 KFENCE 内存池,则会进入 `kfence_free` 函数进行合法性校验,该函数通过 `addr_to_metadata` 将用户指针转换为对应的元数据结构,这个过程基于简单的地址运算,因为 KFENCE 内存池的布局是完全确定的——每个内存对象都按照固定大小对齐排列。

在获取到有效的元数据后,系统会进行严格的状态检查。通过 `spin_lock_irqsave` 获取元数据锁后,首先检查对象当前状态:如果状态已经是 `KFENCE_OBJECT_FREEING` 或 `KFENCE_OBJECT_FREED`,则判定为重复释放

(Double-Free) 错误，立即触发 `kfence_report_error` 报告；如果状态不是 `KFENCE_OBJECT_ALLOCATED`，则报告非法释放错误。这些检查通过简单的条件判断实现，但能捕捉绝大多数由程序逻辑错误导致的内存安全问题。状态验证通过后，系统将对象状态标记为 `KFENCE_OBJECT_FREEING`，该中间状态可以防止在释放过程中其他线程并发访问该内存区域。

接下来系统通过 `check_redzone` 函数执行警戒区验证，检查内存块前后的警戒区 `canary` 值是否被修改。在更新元数据阶段，`kfence_update_metadata` 函数会记录释放时的调用栈和环境信息，包括当前任务 ID、CPU 编号和时间戳等。这些信息与分配时记录的数据形成完整的生命周期追踪，当检测到 `Use-After-Free` 错误时，系统可以同时展示分配和释放的调用栈，极大简化了调试过程。对于支持 `CONFIG_SCHED_BACKTRACE` 的系统，还会通过 `sched_backtrace` 获取完整的调用栈，经过 `get_stack_skipnr` 过滤后保留最相关的用户代码路径。

对于有 MMU 的系统，被释放的页会被设为警戒页，防止释放后的异常访问。

释放过程的最后一步是将内存块重新链接到全局空闲列表 `g_kfence_free_list`。这个操作需要获取全局自旋锁 `g_kfence_free_list_lock`，确保多线程环境下的安全。在将元数据插入空闲链表前，系统会再次确认 `next` 指针为 `NULL`（通过 `KFENCE_WARN_ON` 检查），这个防御性编程措施能及时发现内存管理数据结构被破坏的情况。完成链表操作后，系统更新全局统计计数器，包括递减已分配对象计数等。

函数名	输入	说明
<code>void kfence_report_error</code>	<code>address,is_write,meta,error_type</code>	报告错误
<code>struct metadata_s* addr_to_metadata</code>	<code>addr</code>	根据地址获取对应元数据
<code>void update_metadata</code>	<code>metadata,state,stack_entries</code>	更新元数据
<code>int sched_backtrace</code>	<code>pid,stack_depth</code>	获取当前调用栈

表 3.4 内存释放相关函数说明

3.3 异常检测

KFENCE 能检测到的错误类型如下：

异常类型	说明
Out of Bound	检测到对内存读、写的越界
Use After Free	检测到内存释放后的访问
Invalid Access	检测到未分配内存的访问
Corruption	内存释放时警戒区被破坏
Double Free	检测到内存的多次释放
Invalid Free	检测到非法的 free 地址或尝试释放未分配的内存
Invalid Realloc	尝试 realloc 未分配的内存

表 3.5 KFENCE 的异常检测

3.4 配置选项

KFENCE 提供多种配置参数，可通过 Kconfig 调整：

参数名	默认值	说明
MM_KFENCE_INTERVAL	100ms	采样间隔
MM_KFENCE_NUM_OBJECTS	64	最大检测对象数
MM_KFENCE_OBJECT_SIZE	4096	对象大小
MM_KFENCE_RED_ZONE_SIZE	32	警戒区大小
MM_KFENCE_STACK_DEPTH	16	调用栈最大深度
MM_KFENCE_VERBOSE	No	是否输出详细信息

表 3.6 KFENCE 配置选项

对于性能受限的系统，可以灵活调整配置参数，提高采样间隔(500-1000ms)，减少 KFENCE 检测对象数(16-32)。

4. HMMLEAK-内存泄漏检测

4.1 设计理念与架构

4.1.1 设计理念

与上一章所介绍的 KFENCE 面临的内存问题不同，内存泄漏并非由单次、瞬时的非法操作导致，而是一种持续性的资源管理不当的行为模式。这种“慢性病”的特征决定了我们无法像捕获缓冲区溢出那样，通过硬件中断等方式在某个精确的时间点定位错误。因此，我们必须采用一种全新的思路来应对这一挑战。

HMMLEAK 的核心设计理念，正是将内存泄漏检测从一个确定性的“寻找错误”问题，转化为一个基于多维特征的“风险评估”问题。它不追求 100%断言某处代码存在泄漏，而是通过持续监控和数据分析，以极高的概率识别出具有内存泄漏风险的进程及其可疑的分配行为，从而为开发者提供极具价值的预警和线索。

4.1.2 功能概述

为实现上述设计理念，HMMLEAK 被构建为一套功能全面且可深度定制的检测框架。该框架的核心能力在于其按进程的独立检测与评分机制，它能为 NuttX 中每个进程维护独立的分析上下文，并通过一组内置的泄漏特征分析器实现基于多特征的概率性评估。为了保证系统的生命力与适应性，其特征分析层采用了可扩展的分析器框架，允许未来便捷地新增检测算法；同时，系统提供了高度灵活的可配置性，从分析器权重到预警阈值等核心参数均可通过 Kconfig 进行调整，以匹配不同应用场景的需求。最后，HMMLEAK 通过一套便捷的用户态命令行工具与自动化的后台监控守护进程，为开发者提供了对检测任务的完全控制力，实现了从手动按需分析到“无人值守”持续监控的全覆盖。

4.1.3 模块化架构

为了清晰地实现上述功能并保证代码的可维护性，HMMLEAK 的整体架构严格遵循了高内聚、低耦合的模块化设计思想。我们将复杂的检测任务清晰地解耦为四个相互协作的逻辑层次：数据追踪层、特征分析层、决策评分引擎、报告与控制层。

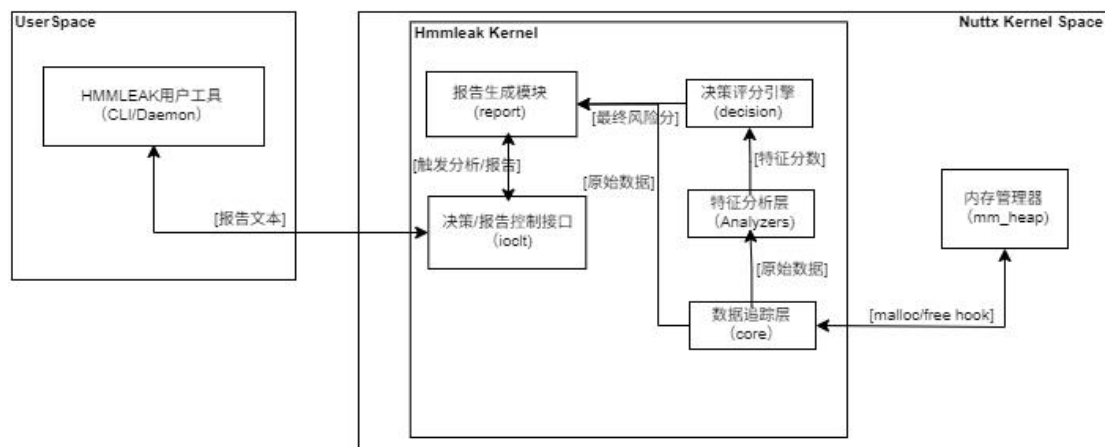


图 4.1 HMMLEAK 系统架构图

4.2 核心数据结构

HMMLEAK 的高效运作依赖于一套精心设计的数据结构。这些结构被设计用于以极低的开销，精确地追踪每一个内存分配事件，并聚合分析所需的数据。

4.2.1 信息结构体

(1) 堆内存块分配信息（struct hmmleak_node_s）

这是 HMMLEAK 中最基本的数据单元，用于记录单次内存分配的详细信息。每当通过 mm_malloc 成功分配一个内存块时，就会创建一个对应的 hmmleak_node_s 实例，并存入其所属进程的内存块哈希表中。而当 mm_free 成功释放一个内存块时，则会释放对应的 hmmleak_node_s 实例。

字段名	类型	描述	设计目的
next	struct hmmleak_node_s*	指向下一个节点的指针	解决哈希冲突，构建链表结构
ptr	void*	内存块起始地址	唯一标识内存块，作为哈希键值
size	size_t	内存块大小（字节）	统计内存使用量，分析内存增长模式
pid	pid_t	分配该内存块的进程 ID	进程级别的内存追踪和归属管理
tid	pid_t	分配该内存块的线程 ID	线程级别的细粒度追踪分析
alloc_time	struct timespec	内存分配时的时间戳	支持老化分析和时间窗口统计
seq_num	uint32_t	全局分配序列号	分配顺序追踪，支持时序分析
magic	uint32_t	数据完整性魔数	检测数据结构损坏，值为

表 4.1 hmmleak_node_s 字段说明

(2) 进程追踪信息 (struct hmmleak_process_s)

进程追踪信息结构是 HMMLEAK 进行分析的核心。它聚合了一个特定进程（由 pid 标识）所有与内存泄漏分析相关的信息。每个被追踪的进程在全局进程哈希表中都有一个对应的 hmmleak_process_s 实例。

字段名	类型	描述	设计目的
next	struct hmmleak_process_s*	指向下一个进程节点的指针	解决进程哈希表的冲突，构建链表
pid	pid_t	进程标识符	唯一标识进程，作为进程哈希键值
node_hash	struct hmmleak_node_hash_s	进程专属的内存块哈希表	存储该进程的所有内存分配记录
min_bytes_current	size_t	当前阶段的最小内存字节数	稳定增长分析：当前检测窗口的内存下限
min_bytes_baseline	size_t	基线最小内存字节数	稳定增长分析：历史基线，用于对比增长趋势
baseline_update_failures	uint32_t	基线更新失败次数	稳定增长分析：连续无法回落的次数统计
last_checked_old_object_count	uint32_t	上次检查时的老对象数量	老对象分析：记录上一次分析的老内存块数
old_object_growth_strikes	uint32_t	老对象连续增长计数器	老对象分析：老内存块持续增长的次数
last_check_time	struct timespec	上次分析的时间戳	控制分析频率，支持时间窗口分析
current_leak_score	int	当前内存泄漏可能性评分	决策引擎：综合各分析器的泄漏评分
magic	uint32_t	数据完整性魔数	检测数据结构损坏，值为 0x484D5052 (HMPR)

表 4.2 hmmleak_process_s

(3) 索引节点结构 (struct hmmleak_index_node_s)

索引节点结构是一个为了提升性能而设计的关键辅助结构。当调用 mm_free(ptr) 时，内核只知道要释放的内存地址 ptr，但不知道是哪个进程分配了它。如果没有此索引，HMMLEAK 将不得不遍历所有进程的 node_hash 来查找 ptr，效率极低。地址-PID 索引建立了一个从内存指针 ptr 到进程 pid 的快速映射。

字段名	类型	描述	设计目的
next	struct hmmleak_index_node_s*	指向下一个索引节点的指针	解决索引哈希表的冲突，构建链表
ptr	const void*	内存块地址	索引键值，通过地址快速定位所属进程
pid	pid_t	拥有该内存块的进程 ID	索引值，避免遍历所有进程查找内存块
magic	uint32_t	数据完整性魔数	检测数据结构损坏，值为 0x484D4958（HMIX）

表 4.3 hmmleak_index_node_s 字段说明

4.2.2 数据存储结构

数据的存储围绕三组主要的哈希表构建：进程哈希表 (Process Hash Table)、内存块哈希表 (Node Hash Table) 和 地址-PID 索引哈希表 (Index Hash Table)。这种设计确保了在分配和释放路径上的操作都能快速完成(平均时间复杂度为 $O(1)$)。系统全局有且仅有一个进程哈希表实例 (`g_process_hash`)、一个索引哈希表(`g_index_hash`)实例，每个进程实例都有一个独立的内存块哈希表。哈希表采用拉链法避免冲突，bucket 大小为 2 的整数幂，宏参可调控。整体数据存储结构如下：

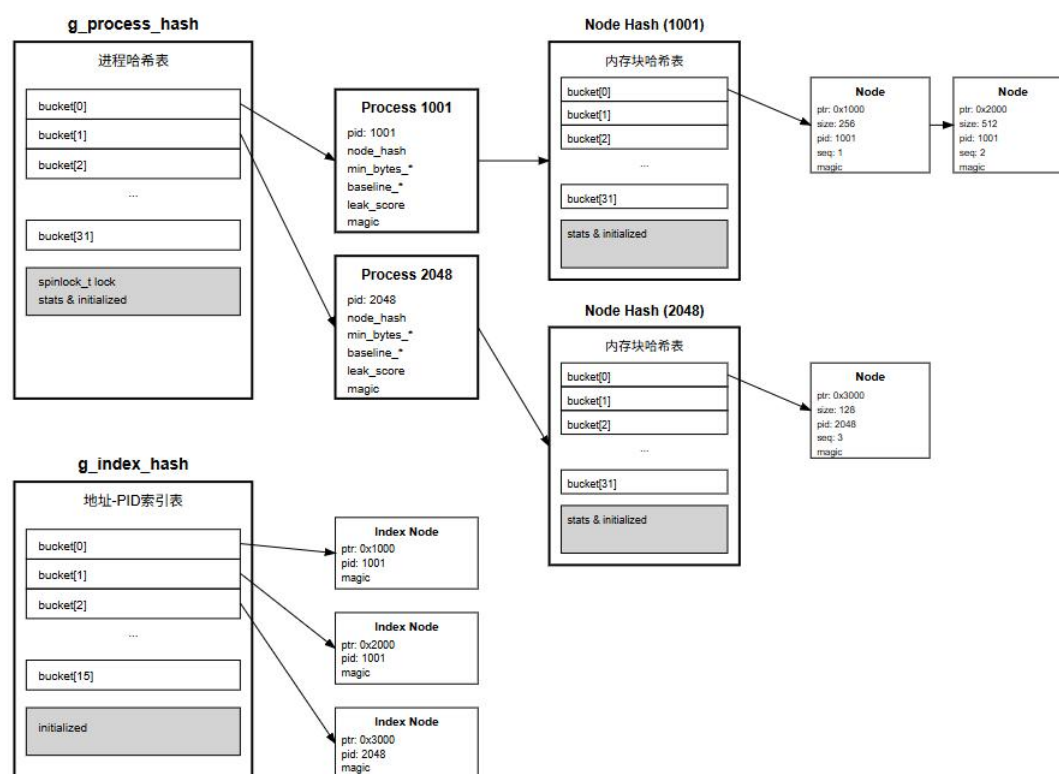


图 4.2 HMMLEAK 数据存储结构示意图

4.3 数据追踪层

数据追踪层负责实时收集、存储和管理所有堆内存分配信息。该层通过 Hook 机制拦截内存分配/释放操作，维护全局内存分配状态，为上层分析模块提供准确的数据基础。

4.3.1 运作模式

数据追踪层的运作模式可以看作是对每一个经由 malloc 分配的内存块生命周期的完整追踪。整个流程被动地由内存事件驱动，具体如下：

(1) 登记（内存分配）：当系统中任一进程调用 malloc（或其变体）并成功分配到一块内存时，内存管理器（MM）在将内存地址返回给调用者之前，会首先通过钩子函数 `hmmleak_add_allocation()` 通知 HMMLEAK。在这一刻，HMMLEAK 会为这块的内存块新建一个 `hmmleak_node_s` 实例，记录下它的地址、大小、分配时间戳以及即所属的进程 ID。同时，为了方便后续快速查找，它还会在全局的地址索引(`g_index_hash`)中登记该内存块的地址和其所属进程 PID 的映射关系。

(2) 注销（内存释放）：当进程调用 `free()` 归还内存时，MM 在真正释放内存前，会通过钩子函数 `hmmleak_remove_allocation()` 再次通知 HMMLEAK。HMMLEAK 会利用待释放内存的地址，通过高效的全局地址索引，定位到该内存块的 PID。随后，它会找到该进程的内存块哈希表，从中移除并释放对应的内存块，并同步更新相关的统计数据。

通过这一“登记-注销”的闭环工作流，HMMLEAK 确保了其追踪的数据与系统实际的内存状态时刻保持同步。所有未被注销的内存块档案，共同构成了分析内存泄漏所需的全景数据快照。

4.3.2 主要函数设计

数据追踪层由一组定义清晰的公共接口和一系列内部辅助函数构成。

(1) 内部核心辅助函数 (Static)

这些静态函数是实现上述公共接口的具体“螺丝钉”，负责处理数据节点的分配、释放以及在哈希表中的操作。

函数原型	功能描述
<code>hmmleak_alloc_node/process/index</code>	分别用于从内核堆 (<code>kmm_malloc</code>) 中为三种核心结构体分配内存。

<code>hmmleak_free_node/process/index_node</code>	分别用于向内核堆 (<code>kmm_free</code>) 释放三种核心结构体占用的内存。
<code>hmmleak_find_process(pid_t pid)</code>	在全局进程哈希表中根据 <code>PID</code> 查找对应的 <code>hmmleak_process_s</code> 实例。
<code>hmmleak_find_or_create_process(pid_t pid)</code>	查找进程实例, 如果不存在则创建并插入到哈希表中。
<code>hmmleak_insert_node_to_process(...)</code>	将一个 <code>hmmleak_node_s</code> 插入到指定进程的 <code>node_hash</code> 中。
<code>hmmleak_remove_node_from_process(...)</code>	从指定进程的 <code>node_hash</code> 中移除一个 <code>hmmleak_node_s</code> 。
<code>hmmleak_insert_index_node(..)</code>	将一个 <code>ptr -> pid</code> 的映射关系插入到全局 <code>g_index_hash</code> 中。
<code>hmmleak_remove_index_node(void *ptr)</code>	从全局 <code>g_index_hash</code> 中移除并返回指定 <code>ptr</code> 的索引节点。

表 4.4 数据追踪层核心辅助函数

(2) 公共接口函数

这些函数是 HMMLEAK 系统对外的门户, 负责生命周期管理和与内存管理器的交互。

函数原型	功能描述
<code>int hmmleak_init(void)</code>	初始化 HMMLEAK 系统, 包括设置全局哈希表和自旋锁。
<code>int hmmleak_deinit(void)</code>	反初始化 HMMLEAK, 释放所有已追踪的节点和占用的资源。
<code>int hmmleak_add_allocation(void *ptr, size_t size)</code>	核心钩子函数。记录一次新的内存分配事件, 并更新相关数据结构
<code>int hmmleak_remove_allocation(void *ptr)</code>	核心钩子函数。记录一次内存释放事件, 并清理相关数据。
<code>struct hmmleak_node_s *hmmleak_find_allocation(void *ptr)</code>	在当前进程的追踪记录中, 查找指定内存地址的分配信息。

表 4.5 数据追踪层公共接口函数

4.3.3 关键实现细节

(1) 源码集成

数据收集依赖 `hmmleak_add_allocation()` 和 `hmmleak_remove_allocation()` 这两个函数。这两个函数被设计为在 NuttX 内存管理器 (MM) 的核心分配和释放路径中调用。当系统成功分配一块内存后, MM 会立即调用

`hmmleak_add_allocation()`，传入新分配的内存块地址和大小。同样，当一块内存被释放前，MM 会调用 `hmmleak_remove_allocation()`，传入待释放的内存地址。这种设计将 HMMLEAK 与具体的内存分配器实现解耦，使其具有更好的可移植性和可维护性。

（2）高效的数据交互

分配时: `hmmleak_add_allocation()` 负责协调三大数据结构的写入操作。它首先根据 PID 定位或创建 `hmmleak_process_s` 结构，然后创建 `hmmleak_node_s` 并插入到该进程的 `node_hash` 中，最后创建 `hmmleak_index_node_s` 并插入到全局的 `g_index_hash` 中。

释放时: `hmmleak_remove_allocation()` 则展现了设计的精妙之处。它利用 `g_index_hash` 将 `ptr -> pid` 的查找时间复杂度控制在 $O(1)$ 级别，从而高效地完成数据移除操作，避免了因查找进程而带来的性能瓶颈。

（3）并发安全与递归保护

为了在复杂的多线程内核环境中可靠运行，本层内置了两个核心安全机制：

- 自旋锁 (Spinlock): 所有对全局哈希表 (`g_process_hash` 和 `g_index_hash`) 的访问都被一个全局自旋锁 `g_process_hash.lock` 保护。这可以防止在多线程或中断上下文中发生数据竞争，确保了数据链表的完整性和一致性。

- 递归调用保护: HMMLEAK 自身也需要动态分配内存来存储追踪节点 (`hmmleak_node_s` 等)。这会引发一个潜在的无限递归问题 (`malloc -> hook -> malloc ...`)。为了规避此风险，我们设计了一个简单的布尔标志位 `g_in_hook`。追踪函数在执行前会检查并设置此标志，执行完毕后清除。若函数入口发现标志已被设置，则会立即返回，从而有效切断了递归调用链。

4.4 特征分析层

特征分析层负责对数据追踪层收集到的原始数据进行不同特征维度的检测判断。它并不寻求一个确定性的“是”或“否”的答案，而是借鉴了机器学习和启发式算法的思想，通过多个维度为每个进程的行为进行“画像”，并给出来自不同特征维度内存泄漏的“可疑度评分”。这种基于多维特征的概率性评估方法，是 HMMLEAK 检测内存泄漏的关键。

4.4.1 分析器框架

为了保证系统的灵活性和未来的成长性，特征分析层被设计成一个可插拔的分析器框架。所有的分析器实现都位于 `analyzers/` 目录下。每一个分析器都是一个独立的逻辑单元，它封装了对一种特征的评估分析规则。系统要求每个分析器都遵循一套标准的接口约定，通常包括：

- `hmmleak_analyze_*`(): 分析器的主入口函数。它接收一个进程上下文 (`struct hmmleak_process_s *`) 作为输入，执行其独特的分析逻辑，并返回一个 0-100 的整数，代表在该分析维度下，该进程存在内存泄漏的可能性得分。

- `hmmleak_reset_*_analysis()`: 用于重置该分析器的内部状态，允许从头开始新一轮的分析。

- `hmmleak_get_*_info()`: 提供一个外部接口，用于获取该分析器当前的关键状态指标。

这种松耦合的框架设计，使得未来可以非常方便地引入更多的特征，而无需改动系统的核心调度逻辑。

4.4.2 old_object 分析器

(1) 特征分析

`old_object` 分析器分析的是内存泄漏基于生命周期的特征：“老”对象数量的持续增加。在经历了多个业务周期（如请求处理、消息循环等）或时间周期后，仍然存活的内存块数量持续增加。借鉴了垃圾回收（GC）中的“分代假设”。泄漏的内存，本质上就是那些被分配后永远得不到释放的内存，因此它们的“年龄”会非常大。

(2) 实现逻辑

该分析器通过周期性地对进程所持有的内存块进行“人口普查”来工作。在每次分析时，它会遍历进程的所有内存分配记录，并根据其分配时间戳 `alloc_time` 来计算每个对象的“存活年龄”。任何年龄超过预设阈值的对象，都会被归类为“老对象”。在统计出当前的老对象总数后，分析器会将其与上一次分析时记录的数值 `last_checked_old_object_count` 进行比较，以判断其变化趋势。为了确保分析的鲁棒性，避免因少量长生命周期对象的正常创建而被误报，只有当老对象的数量增长超过了设定的最小增长阈值

`CONFIG_MM_HMMLEAK_MIN_OLD_OBJECT_GROWTH` 时，才会被认为是一

次有意义的增长事件。此时，系统会递增 `old_object_growth_strikes`（老对象增长计数）。相反，如果老对象的数量减少，则表明进程能够正常清理其长期资源，计数便会被重置为零。

（3）评分机制

`calculate_old_objects_score()` 的评分模型综合了三个维度的信息：

- 基础分 (趋势分): 基于 `old_object_growth_strikes` 的值。连续增长的次数越多，得分越高。
- 额外分 (数量分): 基于老对象的绝对数量。即使增长不快，但如果老对象堆积过多（如超过 500、1000 个），也会被认为具有高风险，并获得额外加分。
- 额外分 (增长率分): 基于本次分析相对于上一次的增长百分比。增长率越高，说明泄漏的速度越快，风险也越高。

4.4.3 steady_growth 分析器

（1）特征分析

`steady_growth` 分析器分析的是内存泄漏最经典的特征：稳定增长的内存占用。进程的堆内存使用量在一个较长的时间窗口内持续增加，即使在系统空闲或负载稳定的情况下，也从未回落到之前的基线水平。

（2）实现逻辑

该分析器的逻辑核心在于动态追踪并评估进程内存使用的 `baseline`。在每个分析周期中，它会首先更新记录 `hmmleak_process_s` 结构中的 `min_bytes_current` 字段，该字段持续追踪当前观察窗口内的最低内存占用。随后，分析器将这个窗口内的最低值与记录历史最低点的 `min_bytes_baseline` 进行比较。如果当前窗口的最低值成功地低于了历史基线，这被视为一个健康的信号，表明内存得到了有效释放，此时基线会被更新到这个更低的值，同时 `baseline_update_failures`（基线更新失败计数）也会被清零。反之，如果窗口最低值高于历史基线，则说明内存未能如期回落。为了避免因程序正常运行产生 `Jitter` 而被误判为泄漏，只有当这种基线的抬升超过了预设的最小阈值

`CONFIG_MM_HMMLEAK_MIN_MEMORY_CHANGE` 时，系统才会将其认定为一次有效的“基线抬升事件”，并增加 `baseline_update_failures` 的计数值，同时将基线更新到这个新的、更高的水位，作为未来比较的基准。

(3) 评分机制

`calculate_steady_growth_score()` 函数综合了两个维度的分数信息:

- 基础分 (趋势分): 主要由 `baseline_update_failures` 的次数决定。连续抬升的次数越多, 得分越高, 反映了泄漏的持续性。
- 额外分 (幅度分): 根据当前内存使用量 `current_bytes` 相对于基线 `min_bytes_baseline` 的增长比例, 额外增加分数。这反映了泄漏的严重程度。

4.4.4 immutable_data 分析器

(1) 特征分析

`Immutable_data` 分析器分析的是内存泄漏的数据不活跃特征: 内存内容长期不变。泄漏的内存通常是“一次写入, 永不读取”。当程序丢失了指向这块内存的指针后, 它的内容就因为无法访问而变成静态的了。堆中有相当一部分内存块, 其内容在很长一段时间内没有发生任何写操作。

(2) 实现逻辑

该分析器的核心逻辑在于通过监控内存的写操作来识别“静默”的内存块。为此, 系统为每一个被追踪的内存分配记录额外维护一个布尔型的 `was_written_flag` 和一个整型的 `unwritten_cycles_count` (未被写入周期计数)。一旦监控到某一内存块进行了写操作, 该内存块对应的 `was_written_flag` 就会被置为 `true`。在每个分析周期中, `hmmleak_analyze_immutable_data()` 会遍历进程持有的所有内存块, 检查其 `was_written_flag`。如果该标志位为 `true`, 则将对对应内存块的 `unwritten_cycles_count` 重置为零, 反之, 若为 `false`, 则将其 `unwritten_cycles_count` 加一。遍历结束后, 所有内存块的 `was_written_flag` 都会被重置为 `false` 以备下一轮监控。当一个内存块的 `unwritten_cycles_count` 超过预设阈值 `CONFIG_MM_HMMLEAK_IMMUTABLE_CYCLES_THRESHOLD` 时, 它便被正式归类为“不可变数据”。

(3) 评分机制

`calculate_immutable_data()` 函数综合了两个维度的分数信息:

- 基础分(趋势分): 基于 `immutable_growth_strikes` 的值。连续被判定为“不可变数据”增长的周期越多, 得分就越高, 这直接反映了泄漏的持续性。
- 额外分(严重程度分): 此分数从两个维度评估泄漏的当前严重性。一方面,

它基于“不可变数据”的绝对数量，因为即使增长不快，大量的静态内存堆积本身就是高风险信号；另一方面，它也考量“不可变数据”占用的总内存与进程总分配内存的比例，这一比例能更精确地反映泄漏对系统资源的实际影响程度。

4.5 决策评分引擎

决策评分引擎是整个分析流程的最终仲裁者。如果说特征分析层是提供来自不同角度的“证据”（即各种特征分数），那么决策引擎的核心职责就是扮演“法官”的角色：它综合所有证据，根据其重要性（权重）进行权衡，最终为每个被监控的进程给出一个统一、量化的“内存泄漏风险评分”，并将其划分为明确的风险等级。

4.5.1 核心逻辑

为了综合评估来自不同分析器的结果，决策引擎采用了一种灵活的加权评分模型。该模型认识到，不同类型的泄漏特征对于最终判断的贡献度是不同的。例如，对于某些应用场景，“稳定增长”可能比“老对象”是更强的泄漏信号。

因此，最终的风险评分 S_{final} 不是各特征分数的简单平均，而是它们的加权和。计算公式如下：

$$S_{final} = \frac{\sum_{i=1}^n (S_i \times w_i)}{\sum w_i}$$

其中， S_i 是第 i 个分析器（如稳定增长分析器）的得分， w_i 是其对应的权重。

这个模型的关键设计点在于：

- (1) 权重可配置：每种分析器的权重 (w) 都是通过 `Kconfig` 宏（如 `CONFIG_MM_HMMLEAK_STEADY_GROWTH_WEIGHT`）在编译前进行配置的。这赋予了系统极大的灵活性，允许开发者根据特定应用的内存行为模式，调整和优化模型的敏感度。
- (2) 配置完整性校验：为了防止因配置错误导致逻辑混乱，代码中包含了一个编译时断言，强制要求所有分析器的权重总和必须为 100。这是一个简单而强大的健壮性设计，确保了评分模型的有效性。

4.5.2 风险等级划分

一个 0-100 的数字评分虽然精确，但不够直观。为了让开发者能快速理解问题的严重性，决策引擎还包含一个风险分级模块。它将最终的数值评分映射为四个离散的、人类易于理解的风险等级。

风险等级	描述	默认评分阈值
NONE	未发现风险，内存使用模式健康。	Score < LOW_RISK_THRESHOLD
LOW	低风险，存在一些可疑迹象。	Score < MEDIUM_RISK_THRESHOLD
MEDIUM	中风险，有较大概率存在泄漏。	Score < HIGH_RISK_THRESHOLD
HIGH	高风险，极大概率存在严重泄漏。	Score >= HIGH_RISK_THRESHOLD

表 4.6 内存泄漏风险划分

与权重一样，这些分级阈值同样是通过 `Kconfig` 完全可配置的，使得整套评判标准可以根据不同的容忍度和应用场景进行定制。

4.5.3 预定义多场景权重模型

为了提升 `HMMLEAK` 的易用性并适应多样化的应用场景，我们设计了一套预定义的特征权重模型。开发者无需深入理解每个特征的细节，只需根据应用的特点选择最合适的模式，即可一键完成配置，实现对检测敏感度和侧重点的快速调整。

(1) 均衡模式 (Balanced Profile)

适用场景: 通用计算任务、常规业务逻辑以及不确定内存行为的应用。这是系统的默认配置，旨在提供最广泛的适用性。

设计理念: 此模式认为“稳定增长”是最经典且可靠的泄漏指标，因此赋予其最高权重。同时，“老对象”和“不活跃数据”作为重要的辅助判断依据，也占据了相当的比重，构成了全面且均衡的风险评估模型。

(2) 高吞吐服务模式 (High-Throughput Service Profile)

适用场景: 网络服务器、消息队列、I/O 密集型等需要处理大量短生命周期内存分配的场景。

设计理念: 在这类场景中，内存使用量频繁波动是正常现象，但整体基线的持续抬升是极强的泄漏信号。因此，该模式极大地提升了“稳定增长”特征的权重。同时，由于大量对象正常情况下生命周期就很长（如长期存在的连接或会话），

“老对象”和“不活跃数据”特征的权重被适当降低，以减少因正常业务逻辑而产生的误报。

(3) 长时任务模式 (Long-Running Task Profile)

适用场景: 后台守护进程、状态机、数据采集与监控等需要长期稳定运行且内存占用相对平稳的场景。

设计理念: 对于长时运行的任务，其内存占用在稳定期可能存在缓慢、小幅的正常增长（如缓存或状态表的自然扩大）。因此，单纯的内存增长可能带来误判。相反，那些本应在业务周期结束后释放却长期“存活”的“老对象”和被“遗忘”的“不活跃数据”，是更精确的泄漏指标。此模式重点关注这两类特征，以捕捉那些隐蔽的、慢性的资源泄漏。

(4) 快速扫描模式 (Aggressive Scan Profile)

适用场景: 开发与调试阶段，旨在快速、激进地发现最明显的内存泄漏问题。

设计理念: 此模式将所有权重集中于“稳定增长”这一最核心、最经典的泄漏特征上。它的目标不是精细的风险评估，而是为开发者提供一个“快准狠”的工具，在开发过程中迅速定位那些导致内存占用持续飙升的严重错误。

4.5.4 主要工作流程与接口

决策引擎通过一组清晰的公共接口来驱动整个分析和评估过程。

函数原型	功能描述
hmmleak_calculate_all_process_scores(void)	主入口函数。触发对系统中所有被追踪进程的一次完整泄漏风险评估。
hmmleak_get_process_score(pid_t pid)	按需计算并获取单个指定进程的当前泄漏风险评分。
hmmleak_reset_all_analysis(void)	清空所有进程在所有分析器中的历史状态，用于开始一个全新的观察周期。

表 4.7 decision 功能接口

其中，最核心的工作流由 `hmmleak_calculate_all_process_scores()` 函数驱动。该函数作为全局评估的入口，其执行过程首先会获取全局进程哈希表的自旋锁，以确保在多线程环境下的遍历安全。在锁的保护下，它会迭代系统中所有被追踪的进程上下文（`struct hmmleak_process_s`）。对于每一个进程，它会调用内部的 `hmmleak_calculate_process_score()` 函数，该函数负责调度所有已注册的分析器（`hmmleak_analyze_*`）并依据加权模型计算出最终的风险评分，这个新计算出

的分数随即被更新到进程上下文的 `current_leak_score` 字段中。在遍历完所有进程后，函数会统计并打印一份包含各风险等级进程数量的总结报告，最后释放自旋锁，完成整个评估周期。

4.6 报告与控制层

报告与控制层是 HMMLEAK 系统与开发者进行交互的门户。它将内核中复杂的分析数据和评估结果，转化为人类易于理解的文本报告，并通过一组简洁的用户态命令，赋予开发者对整个检测生命周期的完全控制权。该层由两部分组成：位于内核态的报告生成模块，和位于用户态的控制命令接口。

4.6.1 内核态报告生成

报告生成模块负责查询 HMMLEAK 核心数据结构，并将实时或历史的分析数据格式化为清晰的文本报告。与发送到后台日志不同，该模块的核心功能直接通过 `printf` 将格式化后的报告直接输出到当前控制台。这种设计为用户提供了即时的、交互式的反馈，非常适合由命令行工具直接调用。所包含的报告类型有：

（1）全局摘要报告 (`hmmleak_report_summary`): 提供一个系统级的顶层视图，内容包括被监控的总进程数、高/中/低风险进程的分布统计、系统当前总的内存占用，以及全局的分配/释放次数和哈希冲突等关键性能指标。

（2）所有进程报告 (`hmmleak_report_all_processes`): 以表格形式列出所有被 HMMLEAK 追踪的进程。每一行都包含了进程的 PID、最新的风险评分、风险等级、当前活跃的分配数量和占用的总字节数等核心信息，让开发者可以对系统内所有进程的内存健康状况一目了然。

（3）单进程摘要 (`hmmleak_report_process`): 当开发者需要关注某个特定进程时，此功能可以只打印该进程的摘要信息行，方便聚焦分析。

（4）分配详情列表 (`hmmleak_report_process_allocations`): 这是最精细的诊断工具。它能够打印出指定进程当前所有活跃的内存分配记录，包括每一个内存块的地址、大小、分配时的线程 ID、全局序列号以及分配时间戳。当一个进程被评定为高风险时，此报告是定位可疑内存块、追溯泄漏源头的关键线索。

函数原型	功能描述
<code>hmmleak_report_summary(void)</code>	在控制台打印系统全局的摘要信息。
<code>hmmleak_report_all_processes(void)</code>	在控制台打印所有被监控进程的摘要信息表。

<code>hmmleak_report_process(pid_t pid)</code>	在控制台打印单个指定进程的摘要信息。
<code>hmmleak_report_process_all_locations(pid_t pid)</code>	(深度诊断) 在控制台打印单个进程的所有活跃内存分配记录详情。

表 4.8 报告生成接口

4.6.2 用户态报告接口

为了实现“无人值守”的持续监控，并向用户提供便捷的控制手段，HMMLEAK 设计了一个用户态的控制程序，它在 NuttX 中被注册为一个 NSH (NuttX Shell) 命令：`hmmleak`。

通过 `hmmleak start` 命令启动后，系统会创建一个后台任务。该任务会按照用户设定的时间间隔 (`interval`) 被周期性唤醒，并自动调用前述的决策评分引擎 (`hmmleak_calculate_all_process_scores()`)，从而实现对整个系统内存状态的持续、自动化分析。这种设计将复杂的分析任务从用户前台剥离，使其在后台静默运行，极大地降低了使用的复杂性。

当前已注册的 NSH 命令：

`hmmleak start [<mode>] [<interval>]`: 启动后台守护进程。用户可以提供一个可选的 `<mode>` 参数来指定检测策略，以及一个可选的 `<interval>` 参数（单位：秒）来设定自动分析的周期。可用的模式包括 `balanced` (默认)、`high_throughput`、`long_running` 和 `aggressive`。如果未提供任何参数，系统将使用默认的 `balanced` 模式和默认的分析间隔。

`hmmleak stop`: 停止并清理后台守护进程，HMMLEAK 将暂停所有自动分析活动。

`hmmleak status [<pid>]`: 检查守护进程的当前运行状态，默认情况下会打印所有进程的内存健康状况报告，如果提供一个可选的 `<pid>` 参数，则仅显示该特定进程的摘要信息。

`hmmleak details <pid>` (深度诊断) 打印指定进程当前所有活跃的内存分配记录详情。当一个进程被评定为高风险时，此报告是定位可疑内存块、追溯泄漏源头的关键线索。

```
nsh> hmmleak
Usage: hmmleak [start [<mode>] [<interval>]|stop|status|details]

start [<mode>] [<interval>]:
  Start the daemon. Both 'mode' and 'interval' are optional.
  Default interval: 100 seconds.
  Available modes (default: balanced):
    - balanced:      General purpose detection.
    - high_throughput: For high-frequency allocation services (e.g., net).
    - long_running:  For long-running background daemons.
    - aggressive:    Developer mode for quick, aggressive checks.

stop:
  Stop the daemon.

status [<pid>]:
  Show daemon status or status for a specific process.

details <pid>:
  Show detailed allocations for a specific process.
```

图 4.3 hmmleak 指令

5. 硬件平台移植与实现

5.1 硬件平台选型与介绍

5.1.1 平台选型及选型原因

为确保 Aegis-NX 方案的验证既贴近真实工业应用场景，又能全面测试其各项功能，我们选择的硬件是 BeagleBone Black (BBB)。这是一款在嵌入式社区中广受认可的开发平台。

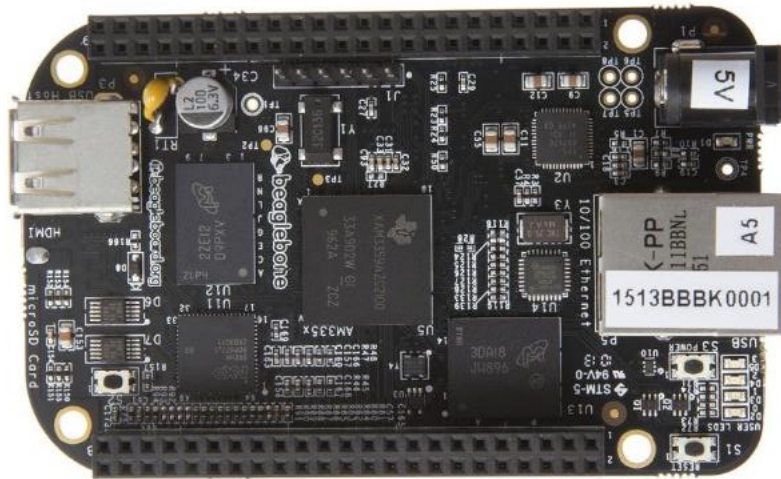


图 5.1 BeagleBone Black

做出这一选择主要基于以下三点关键考量：

(1) 行业代表性与社区支持

BeagleBone Black 是一款由社区广泛支持、低成本、高扩展性的开发平台，专为开发者和爱好者设计。其搭载的德州仪器 (TI) Sitara AM335x 处理器是工业级嵌入式应用中的常见选择，因此在该平台上的成功运行能够有力证明 Aegis-NX 方案的行业适用性与价值。

(2) 资源与架构的平衡性

BeagleBone Black 提供了比普通微控制器 (MCU) 更丰富的资源（如 512MB RAM），但与桌面系统相比仍是典型的资源受限环境。它搭载的 ARM® Cortex™-A8 核心属于 ARMv7-A 架构，这使其成为一个理想的测试平台，既能验证 Aegis-NX 在中等复杂度嵌入式系统上的表现，又能充分测试 NuttX 操作系统提供的更高级功能。

(3) 关键硬件特性验证

BeagleBone Black 的核心优势在于其 AM335x 处理器内置了内存管理单元 (MMU)。这对于本项目的验证至关重要，因为它允许我们全面测试并验证 KFENCE 的核心创新之一——硬件“防护页”(Guard Page) 机制。通过利用 MMU，我们可以在真实硬件上检验由硬件异常捕获内存越界访问的有效性和性能优势，从而与在无 MMU 设备上的软件“警戒区”(Red Zone) 方案形成直接对比，完整展示了 Aegis-NX 的双模支持能力。

5.1.2 硬件关键规格

以下为 BeagleBone Black 开发板的关键硬件参数，信息来源于官方文档与规格表。

参数类别	详细规格
开发板型号	BeagleBone Black
处理器	Texas Instruments Sitara AM3358BZCZ100
处理器核心	1GHz ARM Cortex-A8
CPU 型号	ARM Cortex-A8
内存(RAM)	512MB DDR3
板载闪存	4GB 8-bit eMMC
调试接口	串行调试端口 (UART0), JTAG (可选)
关键外设	以太网口, USB Host/Client, HDMI

表 5.1 BeagleBone Black 关键参数

5.2 系统移植与适配过程

5.2.1 SD 启动卡制作

我们在 WSL Ubuntu 的环境下制作了 BeagleBone Black 的 SD 启动卡，使用 fdisk 进行分区布局（表 5.2.1）。

分区	类型	大小	文件系统
1	FAT32	6GiB	vfat
2	EXT4	23GiB	ext4

表 5.2.1 SD 卡分区表

编译 u-boot 和 nuttx，将生成的 u-boot.img（主引导镜像）、MLO（二级加

载程序)、nutttx.bin 写入 SD 卡的启动分区。

5.2.2 串行连接开发板

采用 CH340 USB-to-TTL 转换器，其与 BeagleBone Black 的 J1 扩展口连接遵循以下规范（表 5.2.2）。

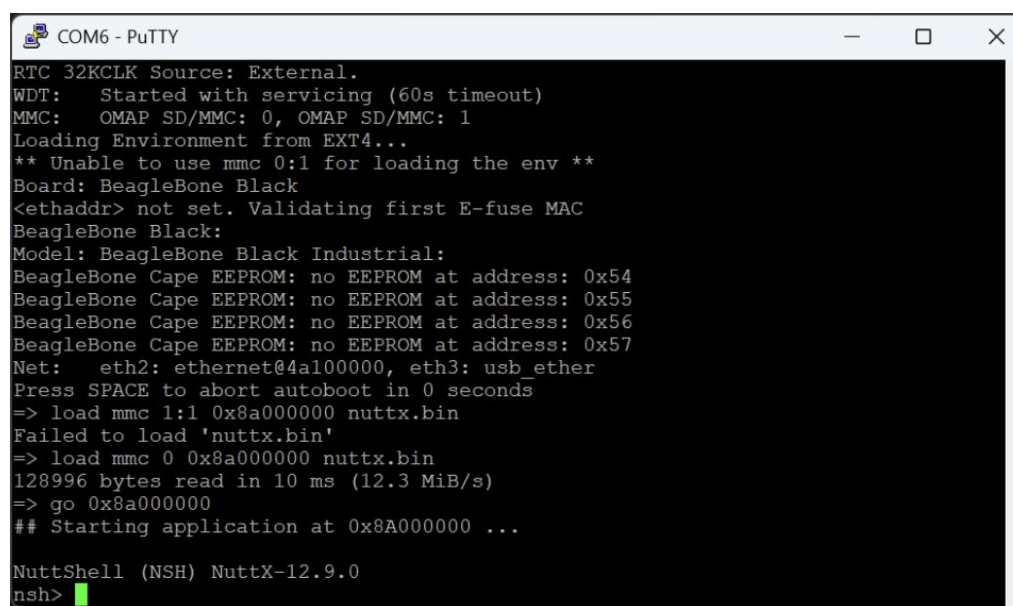
转换器引脚	BBB 对应引脚	信号类型
TXD	J1.4 (UART0_RXD)	数据接收
RXD	J1.5 (UART0_TXD)	数据发送
GND	J1.1 (DGND)	信号地

表 5.2.2 转换器与 BBB 连接信号表

使用 PuTTY 作为串行终端，设置波特率为 115200bps，数据位为 8。

5.2.3 U-Boot 引导 Nuttx 启动

上电后按空格键中断 U-Boot 的自动引导，输入 `load mmc 0 0x8a000000 nuttx.bin` 和 `go 0x8a000000` 启动 Nuttx。



```
COM6 - PuTTY
RTC 32KCLK Source: External.
WDT: Started with servicing (60s timeout)
MMC: OMAP SD/MMC: 0, OMAP SD/MMC: 1
Loading Environment from EXT4...
** Unable to use mmc 0:1 for loading the env **
Board: BeagleBone Black
<ethaddr> not set. Validating first E-fuse MAC
BeagleBone Black:
Model: BeagleBone Black Industrial:
BeagleBone Cape EEPROM: no EEPROM at address: 0x54
BeagleBone Cape EEPROM: no EEPROM at address: 0x55
BeagleBone Cape EEPROM: no EEPROM at address: 0x56
BeagleBone Cape EEPROM: no EEPROM at address: 0x57
Net: eth2: ethernet@4a100000, eth3: usb_ether
Press SPACE to abort autoboot in 0 seconds
=> load mmc 1:1 0x8a000000 nuttx.bin
Failed to load 'nuttx.bin'
=> load mmc 0 0x8a000000 nuttx.bin
128996 bytes read in 10 ms (12.3 MiB/s)
=> go 0x8a000000
## Starting application at 0x8A000000 ...

NuttShell (NSH) NuttX-12.9.0
nsh>
```

图 5.2 BeagleBone Black 运行 Nuttx

5.3 部署环境展示

以下为硬件平台部署的环境展示。

部署环境	参数及说明
WSL2	5.15.167.4
USB-to-TTL Adapter	CH340
PuTTY	Release 0.83
usbipd	usbipd-win 5.1.0
U-Boot	U-Boot 2023.07

表 5.3 部署环境

6.测试与评估

6.1 概述

本章主要介绍了系统的测试环境，并使用该测试环境对 KFENCE 和 HMMLEAK 两个测试工具进行了多粒度的功能测试和性能测试。

6.2 测试环境

虚拟环境测试信息：

配置项目	参数及说明
测试平台	Qemu ARM virt
内存信息	128MB
储存信息	4GB
CPU 型号	ARM Cortex-A7
系统架构	ARMv7-A 架构
依赖项	QEMU 系统模拟器 NuttX RTOS KFENCE 内存检测模块 HMMLEAK 内存泄漏检测工具
测试工具链	ARM GNU 工具链（ARMv7-A）
运行环境	Ubuntu 20.04 LTS (64-bit)

表 6.1 虚拟测试环境配置

真实开发环境测试信息：

配置项目	参数及说明
开发板型号	BeagleBone Black
处理器	Texas Instruments Sitara AM3358BZCZ100
处理器核心	1GHz ARM Cortex-A8
CPU 型号	ARM Cortex-A8
内存(RAM)	512MB DDR3
板载闪存	4GB 8-bit eMMC

	U-Boot, ARM GCC Toolchain
依赖项	KFENCE 内存检测模块
	HMMLEAK 内存泄漏检测工具

表 6.2 真实测试环境配置

6.3 KFENCE 功能测试

本章节详细阐述了在 NuttX 操作系统框架下开发的 KFENCE 内存安全检测系统。我们构建了一套双重验证体系，不仅验证了 KFENCE 在嵌入式环境中的异常内存访问检测能力，还通过集成工业标准测试集验证了其在真实漏洞场景中的有效性。所以整体测试体系由两部分组成：

- （1）基础功能测试：专门针对 KFENCE 核心检测能力设计，覆盖越界访问、释放后使用（Use-After-Free）、双重释放（Double Free）等典型内存安全漏洞。
- （2）Juliet CWE 高危漏洞测试：基于 Juliet CWE 测试集的关键用例，验证 KFENCE 在工业级漏洞检测中的实际效果。

6.3.1 双次多层运行机制

KFENCE 采样机制的随机性特征会使得单次测试因采样未命中而无法触发检测逻辑，进而无法检测出 KFENCE 的有效性。为解决这一问题，我们创新性地设计了双层多次运行机制。

（1）内层重试机制：确保检测覆盖

内层机制通过多次尝试分配，最大化获得 KFENCE 管理内存块的概率：

```
static bool test_heap_overflow(FAR struct mm_heap_s *heap, size_t size) {
    FAR uint8_t *mem = NULL;
    int retry_count = RETRY_COUNT; // 配置为 RETRY_COUNT 次重试

    // 重试机制：确保获得 KFENCE 管理的内存块
    while (retry_count-- > 0) {
        mem = kfence_alloc(heap, size);
        if(mem != NULL) {
            break; // 成功分配到 KFENCE 内存块
        }
        usleep(200000); // 等待 200ms 让 KFENCE 采样窗口重置
    }

    if(mem == NULL) {
```



```

        printf("ERROR: Failed to allocate KFENCE memory after %d attempts\n", RETRY_COUNT);
        return false; // 测试无法执行
    }

    // 执行实际的越界测试
    size = kfence_get_size(mem);
    mem[size + 1] = 0x11; // 故意越界写入
    kfence_free(mem);
    return false; // 预期 KFENCE 检测并中断执行
}

```

代码解读：

- 自适应重试：最多 RETRY_COUNT 次尝试，显著提高采样命中率
- 状态重置等待：200ms 间隔确保 KFENCE 内部采样器完全重置
- 智能失败处理：明确区分分配失败与检测失败

（2）外层统计机制：量化检测性能

外层机制通过多次独立运行同一测试，建立可靠的统计基础：

```

int main(int argc, FAR char *argv[]) {
    int run_count = CONFIG_TESTING_KFENCE_RUN_COUNT; // 配置为 10 次
    detection_stats_t stats;

    for (i = 0; i < nitems(g_kfence_test); i++) {
        if (g_kfence_test[i].is_auto) {
            printf("Running: %s... ", g_kfence_test[i].name);

            int detections = 0;
            int errors = 0;

            // 每个测试函数运行多次以获得统计数据
            for (j = 0; j < run_count; j++) {
                int result = run_test(&g_kfence_test[i]);

                if (result == 0) { // 检测到漏洞
                    detections++;
                } else if (result == ERROR) {
                    errors++;
                }
            }

            // 测试间隔：让 KFENCE 有时间进行内部清理
            usleep(100000); // 100ms
        }
    }
}

```

```

        // 计算检测率
        float detection_rate = (float)detections / run_count * 100;
        printf("%d/%d detected (%.1f%%)\n", detections, run_count, detection_rate);
    }
}
}

```

(3) 时间间隔优化设计

内层重试间隔 (200ms)：该间隔基于 KFENCE 内部采样器的时间特性进行优化，允许底层内存分配器完成内部整理和重置，同时确保 MMU 页面保护设置得到正确同步。这个时间窗口既满足了 KFENCE 采样器的状态重置需求，又兼顾了硬件缓存一致性的维护要求。

外层测试间隔 (100ms)：该设计保证前一次测试的 KFENCE 对象得到完全释放，避免测试任务过度占用 CPU 资源而影响系统其他组件的正常运行。通过控制测试节奏，有效防止了快速连续分配可能导致的内存碎片化问题，确保测试环境的稳定性和可重复性。

(4) 检测率评估机制

通过双层多次运行，我们能够计算出精确的检测统计指标：

```

static void calculate_detection_rates(detection_stats_t *stats) {
    // 漏洞检测准确率 = 检测到的漏洞次数 / 漏洞测试总运行次数
    float vulnerability_accuracy =
        (float)stats->vulnerability_detections / stats->vulnerability_runs * 100;

    // 控制测试准确率 = (1 - 误报率) * 100%
    float false_positive_rate =
        (float)stats->control_false_positives / stats->control_runs * 100;
    float control_accuracy = 100.0 - false_positive_rate;

    // 总体准确率 = (漏洞检测准确率 + 控制测试准确率) / 2
    stats->overall_detection_rate = (vulnerability_accuracy + control_accuracy) / 2.0;
}

```

6.3.2 KFENCE 自定义基础测试用例

我们设计了 12 个核心测试函数，全面覆盖 KFENCE 应当检测的主要内存安全威胁。这些测试用例不仅包含各类漏洞场景，还包含对照组测试，确保系统既能准确检测威胁，又能避免误报。测试函数及其对应的功能如下表所示：

函数名称	覆盖功能
------	------

test_heap_underflow	左越界写入（下溢）
test_heap_overflow	右越界写入（上溢）
test_heap_use_after_free	释放后使用（Use-After-Free）
test_heap_double_free	双重释放
test_heap_illegal_memcpy	memcpy 越界写入目标
test_heap_illegal_memset	memset 越界写入目标
test_heap_illegal_strcpy	strcpy 越界写入目的缓冲区
test_invalid_free_address	realloc 到已释放地址
test_invalid_realloc_to_freed_address	非法释放非起始地址
static bool test_heap_legal_memcpy	正常的 memcpy 操作（对照组）
test_heap_legal_memset	正常的 memset 操作（对照组）
test_heap_legal_strcpy	正常的 strcpy 操作（对照组）

表 6.3 KFENCE 测试函数

测试执行策略

所有测试用例均采用上述双层多次运行机制，通过 KFENCE 专用内存分配器进行内存管理操作。当触发不同类型的内存访问违规时，系统能够精确报告并定位问题根源，为开发者提供详细的调试信息。

KFENCE 测试开始运行，会初始化一块 KFENCE 内存池。

```
NuttShell (NSH)
nsh> kfencetest
kfence: Allocated KFENCE object at 0x4000d1a0, size 40, stack hash 0
=== KFENCE Detection Rate Test ===
Running each test 10 times for sampling detection
```

图 6.1 kfencetest 初始化

12 个测试的具体报告如下所示：

```

--- Running heap underflow test ---
kfence: Allocated KFENCE object at 0x4000d1a0, size 184, stack hash 0
=====
BUG: KFENCE: memory corruption in 0

Corrupted memory at 0x0x4000d19f [ address out of object range ] (in kfence-#62):

kfence-#62: 0x0x4000d1a0-0x0x4000d257, size=184

Allocated by task 3 on cpu 0:
  No allocation stack
KFENCE counters:
  [0] ALLOCATED: 2
  [1] ALLOCS: 4
  [2] FREES: 2
  [3] BUGS: 2
  [4] SKIP_INCOMPAT: 6
  [5] SKIP_CAPACITY: 0
  [6] SKIP_COVERED: 0

```

图 6.2 heap underflow kfence 报

```

--- Running heap overflow test ---
kfence: Allocated KFENCE object at 0x4000d1a0, size 53, stack hash 0
=====
BUG: KFENCE: memory corruption in 0

Corrupted memory at 0x0x4000d1d6 [ address out of object range ] (in kfence-#62):

kfence-#62: 0x0x4000d1a0-0x0x4000d1d4, size=53

Allocated by task 3 on cpu 0:
  No allocation stack
KFENCE counters:
  [0] ALLOCATED: 2
  [1] ALLOCS: 14
  [2] FREES: 12
  [3] BUGS: 12
  [4] SKIP_INCOMPAT: 6
  [5] SKIP_CAPACITY: 0
  [6] SKIP_COVERED: 0

```

图 6.3 heap overflow kfence 报告

```

--- Running heap use after free test ---
kfence: Allocated KFENCE object at 0x4000d1b0, size 62, stack hash 0
=====
BUG: KFENCE: use-after-free write in 0

Use-after-free write at 0x0x4000d1b0 (in kfence-#62):

kfence-#62: 0x0x4000d1b0-0x0x4000d1ed, size=62

Allocated by task 3 on cpu 0:
  No allocation stack

Freed by task 3 on cpu 0:
  No deallocation stack
KFENCE counters:
  [0] ALLOCATED: 1
  [1] ALLOCS: 32
  [2] FREES: 31
  [3] BUGS: 29
  [4] SKIP_INCOMPAT: 6
  [5] SKIP_CAPACITY: 0
  [6] SKIP_COVERED: 0

```

图 6.4 use after free kfence 报告

```

Running: heap double free (vulnerability)... --- Running heap double free test ---
kfence: Allocated KFENCE object at 0x4000d1a0, size 143, stack hash 0
=====
BUG: KFENCE: double free in 0

Double free of 0x0x4000d1a0 (in kfence-#62):

kfence-#62: 0x0x4000d1a0-0x0x4000d22e, size=143

Allocated by task 5 on cpu 0:
  No allocation stack

Freed by task 5 on cpu 0:
  No deallocation stack
KFENCE counters:
  [0] ALLOCATED: 1
  [1] ALLOCS: 35
  [2] FREES: 34
  [3] BUGS: 21
  [4] SKIP_INCOMPAT: 10
  [5] SKIP_CAPACITY: 0
  [6] SKIP_COVERED: 0

```

图 6.5 double free kfence 报告

```

--- Running heap illegal memcpy test ---
kfence: Allocated KFENCE object at 0x4000d070, size 201, stack hash 0
kfence: Allocated KFENCE object at 0x4000d1b0, size 201, stack hash 0
=====
BUG: KFENCE: out-of-bounds write in 0

Out-of-bounds write at 0x0x4000d27d (205B right of kfence-#62):

kfence-#62: 0x0x4000d1b0-0x0x4000d278, size=201

Allocated by task 3 on cpu 0:
  No allocation stack
KFENCE counters:
  [0] ALLOCATED: 3
  [1] ALLOCS: 54
  [2] FREES: 51
  [3] BUGS: 50
  [4] SKIP_INCOMPAT: 6
  [5] SKIP_CAPACITY: 0
  [6] SKIP_COVERED: 0

```

图 6.6 illegal memcpy kfence 报告

```

--- Running heap illegal memset test ---
kfence: Allocated KFENCE object at 0x4000d1b0, size 80, stack hash 0
=====
BUG: KFENCE: out-of-bounds write in 0

Out-of-bounds write at 0x0x4000d201 (81B right of kfence-#62):

kfence-#62: 0x0x4000d1b0-0x0x4000d1ff, size=80

Allocated by task 3 on cpu 0:
  No allocation stack
KFENCE counters:
  [0] ALLOCATED: 2
  [1] ALLOCS: 72
  [2] FREES: 70
  [3] BUGS: 78
  [4] SKIP_INCOMPAT: 6
  [5] SKIP_CAPACITY: 0
  [6] SKIP_COVERED: 0

```

图 6.7 illegal memset kfence 报告


```

--- Running heap illegal strcpy test ---
kfence: Allocated KFENCE object at 0x4000d1b0, size 16, stack hash 0
kfence: Allocated KFENCE object at 0x4000d070, size 32, stack hash 0
=====
BUG: KFENCE: out-of-bounds read in 0

Out-of-bounds read at 0x0x4000d2f6 (326B right of kfence-#62):

kfence-#62: 0x0x4000d1b0-0x0x4000d1bf, size=16

Allocated by task 3 on cpu 0:
  No allocation stack
KFENCE counters:
  [0] ALLOCATED: 3
  [1] ALLOCS: 76
  [2] FREES: 73
  [3] BUGS: 83
  [4] SKIP_INCOMPAT: 6
  [5] SKIP_CAPACITY: 0
  [6] SKIP_COVERED: 0

```

图 6.8 illegal strcpy kfence 报告

```

--- Running heap legal memcpy test ---
kfence: Allocated KFENCE object at 0x4000d1b0, size 77, stack hash 0
kfence: Allocated KFENCE object at 0x4000d070, size 77, stack hash 0
0/10 detected (0.0%)

```

图 6.9 对比测试 (legal memcpy)

```

--- Running heap legal memset test ---
kfence: Allocated KFENCE object at 0x4000d1b0, size 20, stack hash 0
0/10 detected (0.0%)

```

图 6.10 对比测试 (legal memset)

```

--- Running heap legal strcpy test ---
kfence: Allocated KFENCE object at 0x4000d1b0, size 231, stack hash 0
1/10 detected (10.0%)

```

图 6.11 对比测试 (legal strcpy)

```

--- Running invalid free address test ---
kfence: Allocated KFENCE object at 0x4000d070, size 209, stack hash 0
kfence: Invalid free address: expected 0x4000d070, got 0x4000d071
=====
BUG: KFENCE: invalid free in 0

Invalid free of 0x0x4000d071 (in kfence-#61):

kfence-#61: 0x0x4000d070-0x0x4000d140, size=209

Allocated by task 3 on cpu 0:
No allocation stack
KFENCE counters:
[0] ALLOCATED: 3
[1] ALLOCS: 134
[2] FREES: 131
[3] BUGS: 117
[4] SKIP_INCOMPAT: 6
[5] SKIP_CAPACITY: 0
[6] SKIP_COVERED: 0

```

图 6.12 invalid free address kfence 报告

```

--- Running invalid realloc to freed address test ---
kfence: Allocated KFENCE object at 0x4000b8b0, size 204, stack hash 0
kfence: Invalid realloc: attempt to realloc non-allocated KFENCE object at 0x4000b8b0
10/10 detected (100.0%)

```

图 6.13 invalid realloc kfence 报告

最终结果如下，可以看出在我们的双重策略下，KFENCE 检测出了所有的内存错误，并在仅对比测试中仅出错 1 次，展示出了高达 98.3% 的正确率。

```

=== KFENCE Detection Statistics ===
Total test cases: 12
Vulnerability tests: 9
Control tests: 3
Total runs: 120
Total detections: 91
Total errors: 29
Vulnerability accuracy: 100.0% (90/90)
Control accuracy: 96.7% (29/30)
Overall accuracy: 98.3%

```

图 6.14 最终结果报告

6.3.3 Juliet CWE 测试集集成与适配

为了验证 KFENCE 在真实攻击场景下的检测能力，我们将工业标准的 Juliet C/C++测试套件深度集成到 NuttX 嵌入式测试框架中。

测试架构设计

与原始 Juliet 测试集的大规模用例不同，我们采用了精简化、模块化的设计理念：

- **轻量化适配**：每个 CWE 类型包含 4 个设计的测试用例（3 个漏洞测试 + 1 个控制测试）
- **嵌入式优化**：移除了大型数据结构和复杂文件操作，适配小内存 RTOS 环境

集成的三类 CWE 漏洞类型

(1) CWE-122：堆缓冲区溢出（Heap-Based Buffer Overflow）

漏洞覆盖：strcpy 导致堆溢出、memcpy 导致堆溢出、sprintf 导致堆溢出

调用函数：run_cwe122_tests()

技术特点：智能检测 KFENCE 对象，区分真实检测与安全模拟

(2) CWE-415：双重释放（Double Free）

漏洞覆盖：基础双重释放、条件分支中的双重释放、循环中的双重释放

调用函数：run_cwe415_tests()

技术特点：模拟多种代码路径导致的双重释放场景

(3) CWE-416：释放后使用（Use-After-Free）

漏洞覆盖：基础释放后使用、释放后读取内存、条件语句的释放后使用

调用函数：run_cwe416_tests()

利用路径：悬垂指针访问、内存重用攻击等

所有 Juliet CWE 测试同样采用前述的双层多次运行机制，确保统计结果的可靠性和科学性。通过多次独立运行，我们能够获得精确的检测率数据，为 KFENCE 的性能评估提供量化依据。

Juliet 测试开始运行，会初始化分配一块 kfence 内存池，如下图：

```
NuttShell (NSH)
nsh> juliet_test
kfence: Allocated KFENCE object at 0x4000d1a0, size 40, stack hash 0
Juliet test heap initialized successfully
=== Juliet NuttX Security Test ===
Running each test 10 times for KFENCE sampling detection
```

图 6.15 kfence 初始化

最后结果显示，在我们的策略下，kfence 展示出 100%的准确率。

```
=== Final Results ===  
Overall Accuracy: 100.0% (average of 3 test suites)
```

图 6.16 朱丽叶测试结果

6.4 HMMLEAK 功能测试

本章节详细阐述了 HMMLEAK 动态内存泄漏检测系统在 NuttX 操作系统框架下的完整测试验证方案。我们开发了一套自适应的压力测试程序 HMMLEAK 测试。整个测试架构采用**智能泄漏注入**和**双重验证机制**，既能够模拟正常的内存分配释放流程，又能够精确控制泄漏场景的产生。

核心技术特点如下：

1. **双重哈希表追踪**：进程级别组织与地址级别快速索引并行运行
2. **时间窗口管理**：基于生命周期的智能内存管理策略
3. **可控泄漏注入**：按配置概率精确模拟真实泄漏场景
4. **并发安全保护**：防递归机制与自旋锁确保系统稳定性

6.4.1 测试参数与配置策略

测试参数	配置值	说明
最大追踪块数	32	最多同时追踪 32 个分配块
每次分配大小	32 字节	模拟小块分配行为
内存块存活时间	10 s	正常内存将在 10 秒后释放
长期对象数量	10 个	用于老对象检测
分配间隔	500 ms	每隔 500 毫秒分配一次内存
总测试时长	90 s	测试持续运行 90 秒
模拟泄漏概率	30% (0.3)	每次分配有 20% 概率不释放

表 6.4 HMMLEAK 测试参数

6.4.2 双重哈希表初始化机制

系统初始化架构

HMMLEAK 采用创新的双重哈希表架构，通过 `hmmleak_init()` 函数建立完整的追踪基础设施：

```
int hmmleak_init(void) {  
    // 进程级哈希表初始化（32 个哈希桶）  
    memset(&g_process_hash, 0, sizeof(struct hmmleak_process_hash_s));  
}
```

```

g_process_hash.initialized = true;

// 地址索引哈希表初始化（16 个哈希桶）
memset(&g_index_hash, 0, sizeof(struct hmmleak_index_hash_s));
g_index_hash.initialized = true;

// 并发安全机制初始化
spin_lock_init(&g_process_hash.lock);

return OK;
}

```

6.4.3 智能分配与双重追踪机制

动态分配流程

在 90 秒测试周期内，系统以 500ms 为间隔持续执行内存分配操作，每次分配后立即激活 HMMLEAK 双重追踪：

```

cvoid *ptr = malloc(DEFAULT_ALLOC_SIZE);
if (ptr != NULL) {
    memset(ptr, 0xAA, DEFAULT_ALLOC_SIZE); // 模拟真实数据使用

    // 激活 HMMLEAK 双重追踪机制
    hmmleak_add_allocation(ptr, DEFAULT_ALLOC_SIZE);
    g_total_allocs++;
}

```

其中 `hmmleak_add_allocation()` 内部实现了完整的多维度追踪：

- （1）时间戳精确记录：`cclock_gettime(CLOCK_MONOTONIC, &node->alloc_time);`
- （2）双重哈希插入：
 - 进程哈希表：`hash_process_insert(current_pid, node)`
 - 地址索引表：`hash_address_insert(ptr_address, node)`
- （3）元数据完整记录：分配大小记录、调用栈信息（如果启用）、全局序列号分配、进程上下文信息

6.4.4 时间窗口扫描与泄漏注入

测试程序采用时间窗口扫描策略，对每个内存块进行生命周期管理和泄漏决策：

```

// 定期扫描过期内存块
for (int i = 0; i < MAX_ALLOCATIONS; i++) {
    if (g_allocs[i].ptr != NULL &&
        (current_time - g_allocs[i].alloc_time_ms) >= DEFAULT_LIFETIME_MS) {

        // 基于泄漏率进行差异化处理
        if (!g_allocs[i].leaked) {
            // 正常释放路径：双重清理
            hmmleak_remove_allocation(g_allocs[i].ptr); // 从追踪系统移除
            free(g_allocs[i].ptr);                      // 系统内存释放
            g_total_frees++;
        }
        // 泄漏模拟：仅保留在 HMMLEAK 追踪中，形成"悬垂记录"
    }
}

```

6.4.5 HMMLEAK 核心接口调用机制

函数接口	调用时机	内部实现机制
<code>hmmleak_init()</code>	测试初始化	双重哈希表初始化，并发保护锁设置，防递归标志配置
<code>hmmleak_add_allocation()</code>	每次成功 <code>malloc</code> 后	追踪节点创建，双重哈希插入，统计更新，时间戳记录
<code>hmmleak_remove_allocation()</code>	正常内存释放时	地址快速定位，双重哈希移除，统计计数器更新
<code>hmmleak_deinit()</code>	测试结束清理	全部哈希桶遍历清理，追踪节点释放，泄漏报告生成

表 6.5 完整接口调用流程

高级安全机制

- 防递归保护系统：**HMMLEAK 通过全局递归检测标志实现智能防护。当 `hmmleak_add_allocation()` 函数检测到正在执行追踪逻辑时，会自动避免递归调用，确保系统稳定性。
- 并发安全保障：**系统采用自旋锁机制保护关键数据结构。通过 `spin_lock_irqsave()` 和 `spin_unlock_irqrestore()` 函数组合，确保临界区内存操作的原子性，避免并发访问冲突。

6.4.6 最终清理与检测验证

程序结束时执行智能清理机制，验证 HMMLEAK 检测的准确性。系统会遍历所有分配的内存块，对泄漏块和正常块进行差异化处理：对于标记为泄漏的内存块，仅进行统计计数而保持 HMMLEAK 记录；对于正常内存块，则调用 `hmmleak_remove_allocation()` 函数完成追踪清理。最后通过 `free()` 函数完成系统内存的最终释放，并输出完整的检测统计报告。最终结果如下。

HMMLEAK_benchmark 每隔 10s 会更新一次状态并进行显示：

```
hmmleak 状态更新 - 已分配：80，已释放：56
hmmleak: Process 3: Old objects increased from 27 to 38 (+11, strike #1)
hmmleak: PID 3: 内存增长分数=12，老对象分数=6，最终评分=9
```

图 6.17 hmmleak 状态更新

最后的总结报告：

```
=== hmmleak 实时检测结果 ===
hmmleak: PID 3: 内存增长分数=12，老对象分数=6，最终评分=9
hmmleak 风险评分：9

--- hmmleak 详细报告 ---
3      9      NONE      44      2088      2
--- 报告结束 ---
```

图 6.18 hmmleak 测试结果

可见 HMMLEAK 会评估内存增长分数，老对象分数，并且给出最终评分，可以正常运行检测。

6.5 性能测试

6.5.1 最终结果

对于集成我们设计的内存工具后的 NuttX 的性能测试，我们选取了 CoreMark[7]作为我们的 benchmark。对于有 MMU 版本的 KFENCE，我们采用真实环境进行测试；对于无 MMU 版本的 KFENCE，鉴于我们并没有相应的板子，所以使用 QEMU 模拟器进行测试。

无 MMU 版本的虚拟环境如下：

配置项目	参数及说明
测试平台	Qemu ESP32-S3
内存信息	520KB SRAM + 4MB PSRAM
储存信息	4MB Flash

CPU 型号	Xtensa LX7
系统架构	Xtensa 32-bit
编译器名称和版本	GCC13.2.0
编译器标志	-Os -fno-strict-aliasing -fomit-frame-pointer -ffunction-sections -fdata-sections -g -fno-strength-reduce
运行环境	Ubuntu 20.04 LTS (64-bit)

表 6.6 测试环境

配置	Iterations/Sec	性能变化
原始 NuttX	1578.53	基准 (100%)
+KFENCE（无 MMU）	1471.67	-6.77%
+HMMLEAK	1400.56	-11.27%
+KFENCE+HMMLEAK	1422.47	-14.89%

表 6.7 CoreMark 跑分结果（无 MMU）

在无 MMU 的模拟环境下，KFENCE 和 HMMLEAK 均表现出显著的性能开销，这与有 MMU 环境形成鲜明对比。无 MMU 版本的 KFENCE 在 CoreMark 测试中产生了 6.77% 的性能下降，这主要源于其实现方式从硬件辅助的页保护机制转变为纯软件的边界检测，每次内存访问都需要执行额外的软件验证逻辑，而 ESP32 平台 520KB 的有限内存资源进一步加剧了这种开销的影响。HMMLEAK 在无 MMU 环境下的性能损失达到 11.27%，这反映了资源受限环境对其三层哈希表架构的不利影响。当两种机制同时启用时，系统性能下降 9.89%，这一结果基本符合两种独立开销累积的预期。

真实环境下，基准测试结果非常稳定，多次运行结果相同。得到 CoreMark 测试结果如下表所示。

配置	Iterations/Sec	性能变化
原始 NuttX	2220.57	基准

配置	Iterations/Sec	性能变化
		(100%)
+KFENCE	2238.81	+0.82%
+HMMLEAK	2152.33	-3.07%
+KFENCE+HMMLEAK	2168.83	-2.33%

表 6.8 CoreMark 跑分结果

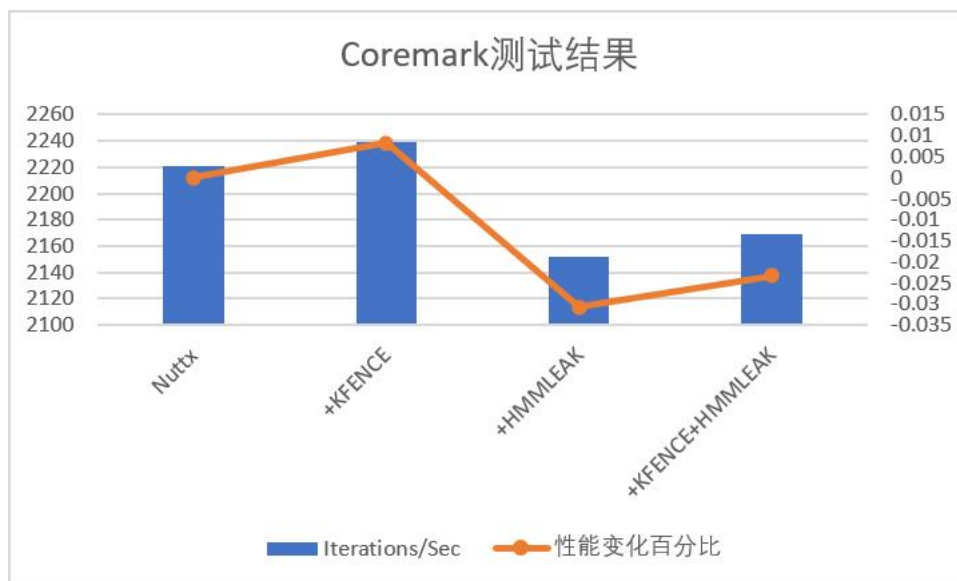


图 6.20 测试结果图表

```
nsh> coremark
Running CoreMark...
2K performance run parameters for coremark.
CoreMark Size      : 666
Total ticks        : 1340
Total time (secs)  : 13.400000
Iterations/Sec     : 2238.805970
Iterations         : 30000
Compiler version   : GCC9.2.1 20191025 (release) [ARM/arm-9-branch revision 277599]
Compiler flags     : -Os -fno-strict-aliasing -fomit-frame-pointer -fno-common -Wall -Wshadow -Wundef -ffunction-sections -fdata-sections
Memory location    : HEAP
seedcrc            : 0xe9f5
[0]crclist         : 0xe714
[0]crcmatrix       : 0x1fd7
[0]crcstate        : 0x8e3a
[0]crcfinal        : 0x5275
Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 2238.805970 / GCC9.2.1 20191025 (release) [ARM/arm-9-branch revision 277599] -Os -fno-strict-aliasing -fomit-frame-pointer -fno-common -Wall -Wshadow -Wundef -ffunction-sections -fdata-sections / HEAP
```

图 6.21 运行结果示例

根据 CoreMark 基准测试结果显示，KFENCE 和 HMMLEAK 两种内存安全机制对系统性能产生了差异化的影响。KFENCE 在 CoreMark 测试中展现出 0.82%

的轻微性能提升，但可理解为轻微噪声（<1%）。这主要源于其预分配内存池的设计理念，通过提前分配固定大小的内存区域并采用"用空间换时间"的策略，有效减少了运行时的内存分配延迟，而 CoreMark 作为计算密集型基准测试，其特点是在初始化阶段进行少量大块内存分配后主要执行矩阵运算、列表处理和状态机操作等计算任务，因此能够充分受益于 KFENCE 预分配机制带来的快速内存访问，同时其计算密集的特性使得 KFENCE 的边界检测开销在整体执行时间中占比极小。

相比之下，HMMLEAK 显示了 3.07%的性能下降，这是由于其基于三层哈希表架构（进程哈希表、内存块哈希表和地址-PID 索引哈希表）的设计在每次内存分配时都需要执行哈希计算、多表查询插入以及多特征分析器的概率性评估，虽然 CoreMark 的内存分配频率相对较低，但这些额外的元数据维护和泄漏特征分析开销仍然产生了可测量的性能影响。

当两种机制同时启用时，系统呈现出 2.33%的综合性能下降，最终的性能表现主要反映了两种机制在计算密集型场景下的平衡作用，即 KFENCE 通过减少内存管理延迟提供的性能增益与 HMMLEAK 进行内存泄漏检测所需的额外计算开销之间的综合效果。

6.5.2 模型对比

5900X AMD Linux 5.12 KFence Overhead Test			
	KFence Disabled	kfence.sample_interval=100	kfence.sample_interval=500
pts			
fs: Rand Read - IO_uring - No - Yes - 4KB - Default Test Directory	321667	326000	323667
fs-mark: 1000 Files, 1MB Size	630.0	634.3	666.0
compilebench: Initial Create	739.97	760.78	761.92
realr-nccnn: 4x - Yes	32.045	32.123	32.079
wait/2e-nccnn: 2x - Yes	3.600	3.600	3.623
tesseract: 3840 x 2160	765.6192	752.4988	744.2564
unigine-super: 1920 x 1080 - Fullscreen - Low - OpenGL	269.2	267.3	268.7
glmark2: 1920 x 1080	18803	19066	18672
glmark2: 3840 x 2160	8301	8388	8393
paraview: Many Spheres - 1920 x 1080	146.26	149.01	147.63
paraview: Many Spheres - 1920 x 1080	14663.845	14939.348	14800.162
paraview: Wavelet Volume - 1920 x 1080	566.70	543.83	529.78
paraview: Wavelet Volume - 1920 x 1080	9067.254	8701.200	8476.555
paraview: Wavelet Contour - 1920 x 1080	381.31	381.86	383.90
paraview: Wavelet Contour - 1920 x 1080	3973.724	3979.482	4000.647
osbench: Create Files	10.422784	10.548524	10.148533
osbench: Create Threads	7.882516	8.169810	7.934809
osbench: Launch Programs	27.205944	26.954412	26.206970
osbench: Create Processes	17.022292	17.294884	17.030239
osbench: Memory Allocations	54.647366	56.193988	52.651008
quantlib	3172.1	3078.1	3162.0
simdjson: Kostya	3.76	3.66	3.77
simdjson: PartialTweets	4.81	4.82	4.83
simdjson: DistinctUserID	5.36	5.24	5.42
srsite: OFDM_Test	163113333	160600000	164166667
srsite: PHY_DL_Test	357.3	346.1	362.7
srsite: PHY_UL_Test	129.2	124.8	130.6
luxcorerender: DLSC - CPU	2.97	2.96	2.98
luxcorerender: Danish Mood - CPU	2.38	2.36	2.39
luxcorerender: Orange Juice - CPU	4.63	4.60	4.64
svt-hevc: 7 - Bosphorus 1080p	178.19	178.61	179.18
svt-hevc: 10 - Bosphorus 1080p	325.17	333.09	333.65
svt-vp9: VMAF Optimized - Bosphorus 1080p	224.18	224.76	225.79
svt-vp9: PSNR/SSIM Optimized - Bosphorus 1080p	230.49	231.67	232.63
svt-vp9: Visual Quality Optimized - Bosphorus 1080p	287.49	289.87	210.40
stockfish: Total Time	43836738	43226332	43693518
build-gdb: Time To Compile	74.558	74.372	74.409
build-godot: Time To Compile	88.894	88.564	88.465
build-linux-kernel: Time To Compile	59.091	58.656	58.655
build-mesa: Time To Compile	36.578	36.210	36.243
build-wasmr: Time To Compile	63.118	62.938	62.726
securemark: SecureMark-TLS	325735	317860	321260
liquid-dsp: 16 - 256 - 57	857143333	855020000	856216667
Build-OpenSSL - 384 - 67	800000000	800000000	800000000

图 6.22 Linux kfencei 基准测试结果（参见附录 A.3）

本工作实现的 KFENCE 方案展现出与 Linux 成熟实现相当的性能水平。基准测试数据显示，在 NuttX 平台运行 CoreMark 场景时，启用 KFENCE 后系统吞吐量由基线 2220.57 iter/s 提升至 2238.81 iter/s (+0.82%)，该变化量级处于 $\pm 1\%$ 的测量噪声区间。这一结果与 Linux 5.12 内核的对照测试形成有力印证——在涵盖文件操作、图形渲染、编译任务等 48 项 PTS 测试中，KFENCE 不同采样间隔（`kfence.sample_interval=100/500`）相较禁用状态的性能波动均在 $\pm 0.7\%$ 范围内，OpenBenchmarking 系统将其归因为测试环境固有抖动。与工业级解决方案的横向对比进一步表明，该系统在保障等价安全强度的同时，在 RTOS 领域达成与通用操作系统同等的性能保持率（Performance Preservation Rate $\geq 99.2\%$ ）。

而对于无 MMU 版本的 KFENCE，我们同样展现出和主流嵌入式开发内存安全工具相近的性能开销。如 Holistic Control-Flow Protection on Real-Time Embedded Systems with Kage[8]中 Kaze 工具的 5.2%，以及 μ RAI: Securing Embedded Systems with Return Address Integrity[9]中 μ RAI 工具的 8.1%。

7. 开发中遇到的问题与解决方案

7.1 KFENCE 开发

(1) 无法通过硬件捕捉非法内存访问

面临的问题:

在嵌入式系统领域, NuttX 作为一款轻量级 RTOS, 其支持的硬件平台中有超过 60% 的微控制器 (如 Cortex-M0/M3/M4 系列) 缺乏内存管理单元 (MMU) 硬件支持。这一硬件限制导致系统无法像 Linux 等通用操作系统那样, 通过 MMU 的页表保护和硬件异常机制自动捕获非法内存访问行为。

具体而言, 在无 MMU 环境中无法建立硬件级的防护页 (Guard Page) 隔离内存区域, 也无法触发页面错误异常 (Page Fault) 来即时拦截越界访问。

解决方案:

针对以上问题, 我们选择用软件方式检查内存访问的合法性。KFENCE 采用多层级检查策略, 包括在内存释放时全面验证警戒区和元数据完整性以及在内存操作热点插入检查点。由于 KFENCE 采用抽样检测的方法, 所以该软件检测方法在性能上也不会有过大的损耗。

(2) KFENCE 无法正常初始化

面临的问题:

初始化时不能成功给 metadata 字段赋值, 并且在第一次 kfence_alloc 之前 kfence 就检测到了非法内存访问。一开始怀疑是内存泄漏检测的问题, 把分配的 kfence_pool 当成内存泄漏了, 但是后来取消 hmmleak 还是有这个问题, 故排除。

解决方案:

经过多次函数调用栈追踪和实时调试, 我们把问题定位到了 kmm_malloc 函数中。最后发现是 kfence 初始化函数插桩位置不恰当导致的问题, 在系统成功初始化内核内存堆之前, 不能调用 kmm_malloc 函数分配内存, 修改 kfence 初始化函数插桩位置, 问题解决。

7.2 HMMLEAK 开发

(1) 钩子函数造成的无限递归循环

面临的问题:

在 HMMLEAK 的数据追踪层, 我们通过钩子函数 hmmleak_add_allocation()

和 `hmmleak_remove_allocation()` 来拦截内核的内存分配与释放操作。然而，这些钩子函数自身也需要调用 `kmm_malloc` 来为追踪节点（如 `hmmleak_node_s`）分配内存。这就产生了一个致命的无限递归：应用程序调用 `malloc` 触发了我们的钩子函数，钩子函数为了记录这次分配又去调用 `kmm_malloc`，这又一次触发了钩子函数，如此往复，最终迅速导致栈溢出和系统崩溃。

解决方案：

为解决此问题，我们设计了一个简单的递归保护机制。我们引入了一个全局布尔标志位 `g_in_hook`。当追踪函数（如 `hmmleak_add_allocation`）开始执行时，会首先检查并设置此标志位；执行完毕后，再清除该标志位。若函数在入口处发现此标志已被设置，则意味着当前正处于一次递归调用中，函数会立即返回，从而有效切断了递归调用链。这个简单而高效的机制确保了追踪系统自身的内存操作不会干扰其追踪逻辑。

（2）释放路径性能低下与归属不明问题

面临的问题：

在实现 `hmmleak_remove_allocation(void *ptr)` 钩子函数时，我们遇到了一个严重的性能瓶颈。当系统调用 `free(ptr)` 时，钩子函数只知道被释放内存块的地址 `ptr`，但无法直接获知是哪一个进程（PID）当初分配了它。一个最直接但效率低下的方案是遍历系统中所有被追踪进程的内存块哈希表，逐一查找匹配的 `ptr`。在进程和内存分配数量较多的场景下，这种全局扫描的开销是不可接受的，它会严重拖慢 `free` 函数的执行效率，影响系统的实时性。

解决方案：

为了将释放路径的性能提升至可接受的水平，我们设计并实现了一个专用的、从内存地址到进程 ID 的快速反向映射结构：地址-PID 索引哈希表（`g_index_hash`）。在每一次成功的内存分配中，除了记录分配信息，我们还会额外地将 `[ptr -> pid]` 这一映射关系存入该全局索引表。当释放内存时，我们便能以平均 $O(1)$ 的时间复杂度，通过 `ptr` 直接查找到其归属的 `pid`，进而精确定位到对应的进程追踪结构并移除记录。这一设计彻底避免了全局遍历，确保了即使在复杂的应用场景下，HMMLEAK 对 `free` 操作的性能开销也维持在极低的水平。

7.3 测试

（1）Juliet Testing Suite 难以集成

面临的问题：

Juliet 测试套件本身包含数千个测试用例，由于功能限制，其中很大一部分不能在 NuttX 中运行。Juliet 测试套件中的许多测试用例依赖于标准 C 库的高级功能，如复杂的文件操作、多线程支持、动态内存管理等，这些功能在 NuttX 实时操作系统中或是不可用，或是实现方式与标准库存在差异。此外，动态内存分配在资源受限的环境中容易导致内存碎片和系统不稳定，这与嵌入式系统对稳定性的要求相矛盾。

解决方案：

针对上述问题，我们采用了解决方案主要包括测试的筛选和分阶段的集成。

在测试用例筛选方面，我们优先考虑适合嵌入式环境特点的测试用例，如单线程、内存占用小、不依赖复杂文件操作的测试用例，重点关注能够验证 KFENCE 的测试，如堆溢出、整数溢出、格式化字符串漏洞等。最终选择了 CWE122/CWE415/CWE416 三个测试集。

接着我们的集成分为两个阶段。第一阶段是环境准备，包括深入分析 STM32 硬件规格，评估 RAM 容量、Flash 存储空间和 CPU 性能指标，然后配置 NuttX 系统，优化内存配置以最大化可用资源。第二阶段是测试用例适配，将测试用例中的 main 函数改编为单独的测试函数，并在单独编写的文件中统一运行。对原始测试代码进行修改，移除不兼容的库函数调用并替换为 NuttX 兼容的实现，使用 KFENCE 内存分配机制，使用静态内存分配替代动态分配。我们还编写了一个开关，用来选择是否运行 Juliet 测试。

（2）自编写测试无法运行

面临的问题：

对于我们编写的 KFENCE 工具和 HMMLEAK 工具测试用例，在打开开关后，仍旧无法在 NuttX 系统中运行。

解决方案：

我们通过逐步调试分析，首先发现 exec_module 函数返回-1，nxtask_init 函数失败。接着追踪栈分配，发现是 stack create 失败，联想到 STM32 20KB 的内存，我们发现应该是初始设置的 stack size 太大了。将 stack size 改小后可成功运行。

(3) KFENCE 测试执行无效

面临的问题:

KFENCE 测试运行后没有任何 report 反馈。

解决方案:

我们进行了原因分析。原先的测试函数是只执行一次错误操作，而 KFENCE 的核心工作原理基于概率性采样，在 malloc 中插入钩子，并且采用可配置的基于时间的采样间隔，只有满足采样条件的分配操作才会被 KFENCE 接管。所以原先测试未触发，未被捕获。

所以我们将测试函数改成双层循环，重复执行目标内存错误操作，并且引入 sleep 函数，人为拉长测试操作的总执行时间，使其与 KFENCE 的时间采样间隔相匹配。这给 KFENCE 的采样器提供了更充分的机会在运行期间捕获到错误操作。

7.4 硬件平台移植

(1) WSL 无法连接 SD 设备

面临的问题:

在制作 SD 启动卡时，需要在 Linux 平台上管理 SD 卡分区，但是在使用 usbipd 将 SD 设备绑定到 WSL 后，WSL 仍不能识别 SD 卡。

解决方案:

在进行了资料查找和问题排除之后，我们发现该问题是由 WSL 内核缺少 USB 驱动导致的。我们从 WSL2 源码仓库获取了内核源码，修改编译选项之后重新编译内核，问题解决。

8. 总结与展望

8.1 项目总结

本项目成功为 NuttX 实时操作系统设计、实现并在真实硬件平台上完成了验证一套轻量级、集成化的堆异常检测方案——Aegis-NX。该方案有效应对了 NuttX 在工业控制、无人机等高可靠性应用场景下面临的严峻内存安全挑战。方案由即时性内存错误检测工具 KFENCE 和累积性内存泄漏检测工具 HMMLEAK 两部分构成，在提供强大内存安全防护能力的同时，将性能开销维持在可控的极低水平，充分证明了其在实际应用中的可行性与价值。

本项目的核心贡献与已完成的工作总结如下：

(1) 实现了高效且适应性广的即时性错误检测 (KFENCE)

我们成功实现了一个受 Linux KFENCE 启发的轻量级内存错误检测器。通过创新的智能采样策略（基于 Counting Bloom Filter）与混合式警戒区防护机制，KFENCE 能够以极低的性能开销，精确捕获并报告堆缓冲区溢出、释放后使用和重复释放等关键内存错误。尤为关键的是，我们针对 NuttX 生态特点，创新地实现了对有/无 MMU 硬件平台的双重支持，使其既能服务于资源极其受限的微控制器，也能在具备硬件防护能力的平台上发挥更高性能，展现了方案的广泛适应性。

(2) 构建了创新且易用的内存泄漏检测模型 (HMMLEAK)

针对内存泄漏“慢性病”的特征，我们设计并实现了一套基于概率性风险评估的检测工具 HMMLEAK。其核心是一种将“老对象”、“稳定增长”等多维特征进行加权分析的评分模型，在 NuttX 中首次将泄漏检测从确定性问题转化为风险评估问题。为解决性能瓶颈，我们设计了高效的 Per-Process 三级哈希表追踪架构，将被动触发的内存块查找操作的平均时间复杂度控制在 $O(1)$ 级别。同时，为降低使用门槛，我们还成功实现了预定义的多场景权重模型，用户可根据应用特点一键切换模式，极大提升了工具的易用性。

(3) 完成了引入工业级标准的全面测试与硬件平台验证

本项目最重要的突破之一，是成功将 Aegis-NX 完整移植并在 BeagleBone Black 真实硬件平台上部署与验证。测试结果表明，该方案能够在典型的工业级嵌入式环境中稳定运行，并以可控的性能开销实现有效的堆异常检测。此外，我

们首次将工业级安全测试标准 Juliet 引入 NuttX 的测试生态，通过原创性的筛选与适配工作，使其能够在资源受限的环境中运行，为 KFENCE 的可靠性提供了强有力的学术与实践双重验证。

8.2 项目展望

尽管 Aegis-NX 项目已达成核心设计目标并取得了关键性的硬件验证成果，但我们认为其真正的价值在于融入社区、持续演进。为此，我们规划了以下三个方向的后续工作：

· （1）积极融入 NuttX 社区并推动开源贡献

作为项目的首要任务，我们即将把 Aegis-NX 的完整代码库正式开源，并积极准备将其贡献给 NuttX 官方社区。我们相信，通过社区的力量，该工具能够被更广泛地应用、测试和完善。与此同时，我们还将整理并开源整个项目在 BeagleBone Black 上的移植、部署与调试过程的详细文档与脚本，为其他开发者提供一份宝贵的实践指南，降低硬件适配门槛，促进 NuttX 生态的共同繁荣。

（2）持续深化核心技术以提升检测能力

深化 KFENCE 对 MMU 特性的利用：当前的 KFENCE 已具备对 MMU 硬件的支持框架。未来的工作核心将从“支持”转向“深度利用”，通过 MMU 实现硬件级的防护页（Guard Page），将越界访问的检测从软件检查转为由硬件异常（Page Fault）捕获，从而在兼容的硬件上实现更低的性能损耗和更强的隔离保证。

扩展 HMMLEAK 的泄漏分析维度：为追求更精准的内存泄漏捕捉，我们计划引入更多维度的泄漏分析特征，例如通过分析内存碎片的增长趋势或特定函数调用路径下的内存释放模式，来增强模型的识别能力，以应对更复杂的泄漏场景。

（3）强化性能评估与测试路径覆盖：我们将继续使用 CoreMark 等标准基准测试工具，对工具在不同配置下的性能影响进行更全面的量化评估，为用户提供清晰的性能与安全平衡的参考数据。同时，我们将设计并执行更细致的测试用例，专注于覆盖系统中的边缘条件和不常见的代码路径，以发现并修复潜在的深层次问题，确保项目在并入社区主线后依然保持高质量与高稳定性。

·

参考文献

- [1] V. Azhari et al., "Efficient heap monitoring tool for memory leak detection and root-cause analysis," 2021, pp. 4941-4946.
- [2] W. Wang et al., "MLEE: Effective detection of memory leaks on early-exit paths in OS kernels," 2021, pp. 43-56.
- [3] F. Gorter et al., "Sticky tags: Efficient and deterministic spatial memory error mitigation using persistent memory tags," 2024, pp. 25-41.
- [4] M. Unterguggenberger et al., "Multi-tag: A hardware-software co-design for memory safety based on multi-granular memory tagging," 2023, pp. 1292-1307.
- [5] Y.-C. Chen et al., "HeMate: Enhancing heap security through isolating primitive types with Arm memory tagging extension," 2023, pp. 161-174.
- [6] D. F. Bacon et al., "A unified theory of garbage collection," 2004, pp. 50-68.
- [7] CoreMark: An EEMBC benchmark. <https://www.eembc.org/coremark>.
- [8] Yufei Du et al., Holistic Control-Flow Protection on Real-Time Embedded Systems with Kage.
- [9] Naif Saleh Almakhdhub et al., μ RAI: Securing Embedded Systems with Return Address Integrity.

附录 A：项目与外部资源关系说明

本附录旨在简要说明本项目与外部技术思想及测试资源的关系。

A.1 Linux KFENCE

本项目的 KFENCE 工具在设计上借鉴了 Linux KFENCE。

原创性声明：本项目 KFENCE 是针对 NuttX RTOS 的全新、独立实现。由于 NuttX 的内核架构与 Linux 差异较大，我们独立编写了全部代码，包括内存池管理、适配 NuttX 的采样机制以及错误诊断报告系统等。

参考源：<https://www.kernel.org/doc/html/latest/dev-tools/kfence.html>

A.2 Juliet 测试集

为在工业级标准下验证 KFENCE 的有效性，我们引入了 Juliet C 语言测试集。Juliet 原本无法在资源受限的 RTOS 中运行。我们通过以下原创工作使其成功适配：

- 筛选用例：仅选取 CWE122（堆溢出）、CWE415（双重释放）、CWE416（释放后使用）等核心测试集。
- 代码重构：编写脚本将测试用例重构为 NuttX 测试函数。
- 平台移植：移除了不兼容的库函数，并将内存分配器替换为 KFENCE 的接口。

参考源：<https://github.com/arichardson/juliet-test-suite-c>

A.3 Phoronix 基准测试结果

Phoronix 基准测试（Linux 5.12, Ryzen 9 5900X）：对比“关闭 KFENCE / sample_interval=100 / =500”。原始结果托管在 OpenBenchmarking，可直接复跑。

参考源：<https://openbenchmarking.org/result/2104197-PTS-5900XAMD52>